Case Study Work

Johan Nino Espino

2026-10-24

```
# 1. Load the original data
train_raw <- read.csv("data/data-train.csv")
test_raw  <- read.csv("data/data-test.csv")

train <- train_raw
test  <- test_raw

predictors <- c("St", "Re", "Fr")
raw_moments <- paste0("R_moment_", 1:4)
```

```
# 2. Check required columns
stopifnot(
  all(c(predictors, raw_moments) %in% names(train)),
  all(predictors %in% names(test))
)

# Check that the required columns are numeric and have no missing values.
# Fr = Inf is an intentional simulation setting, not a missing value.
stopifnot(
  all(vapply(train[c(predictors, raw_moments)], is.numeric, logical(1))),
  all(vapply(test[predictors], is.numeric, logical(1))),
  !anyNA(train[c(predictors, raw_moments)]),
  !anyNA(test[predictors])
)

# Re, St, and the raw moments must be finite.
stopifnot(
  all(is.finite(as.matrix(train[c("St", "Re", raw_moments)]))),
  all(is.finite(as.matrix(test[c("St", "Re")]))),
  all(train$Fr > 0),
```

```
  all(test$Fr > 0)
)
```

```
# 3. Convert raw moments into distribution summaries
m1 <- train$R_moment_1
m2 <- train$R_moment_2
m3 <- train$R_moment_3
m4 <- train$R_moment_4

train$mean_volume <- m1
train$variance_volume <- m2 - m1^2

# Stop and investigate if any variance is nonpositive.
if (any(train$variance_volume <= 0)) {
  stop("At least one calculated variance is nonpositive. Inspect those rows.")
}

train$sd_volume <- sqrt(train$variance_volume)

central_moment_3 <- m3 - 3 * m1 * m2 + 2 * m1^3
central_moment_4 <- m4 - 4 * m1 * m3 + 6 * m1^2 * m2 - 3 * m1^4

train$skewness_volume <- central_moment_3 / train$sd_volume^3
train$kurtosis_volume <- central_moment_4 / train$sd_volume^4

response_names <- c(
  "mean_volume", "variance_volume", "sd_volume",
  "skewness_volume", "kurtosis_volume"
)

stopifnot(
  all(is.finite(as.matrix(train[response_names])))
)
```

```
# 4. Print a compact preparation check
preparation_check <- data.frame(
  dataset = c("Training", "Test"),
  rows = c(nrow(train_raw), nrow(test_raw)),
  original_columns = c(ncol(train_raw), ncol(test_raw)),
  missing_entries = c(sum(is.na(train_raw)), sum(is.na(test_raw))),
  duplicate_input_settings = c(
    sum(duplicated(train_raw[predictors])),
```

```
    sum(duplicated(test_raw[predictors]))
  ),
  infinite_Fr_settings = c(
    sum(is.infinite(train_raw$Fr)),
    sum(is.infinite(test_raw$Fr))
  )
)

knitr::kable(preparation_check, caption = "Data preparation checks")
```

Table 1: Data preparation checks

dataset	rows	original_columns	missing_entries	duplicate_input_settings	infinite_Fr_settings
Training	89	7	0	0	30
Test	23	3	0	0	12

```
# Preview the predictors and calculated responses
knitr::kable(
  head(train[c(predictors, response_names)]),
  digits = 5,
  caption = "First six training observations"
)
```

Table 2: First six training observations

St	Re	Fr	mean_volume	variance_volume	sd_volume	skewness_volume	kurtosis_volume
0.10	224	0.052	0.00216	0.13035	0.36103	305.42800	93371.6555
3.00	224	0.052	0.00379	0.47041	0.68586	216.76225	47012.2515
0.70	224	Inf	0.00291	0.04349	0.20854	90.58974	8216.7778
0.05	90	Inf	0.06353	0.08662	0.29431	17.67980	420.2523
0.70	398	Inf	0.00037	0.00622	0.07889	257.58554	66372.7845
2.00	90	0.300	0.14780	1.98496	1.40888	12.64607	165.0999

```
# Summarize the calculated responses
summary(train[response_names])
```

```
  mean_volume       variance_volume       sd_volume         skewness_volume
 Min.   :0.000222   Min.   :1.000e-04   Min.   : 0.01006   Min.   : 11.97
 1st Qu.:0.002157   1st Qu.:2.447e-02   1st Qu.: 0.15643   1st Qu.: 72.55
```

```
 Median :0.002958   Median :8.078e-02   Median : 0.28423   Median :110.12
 Mean   :0.040394   Mean   :9.249e+01   Mean   : 3.65112   Mean   :162.81
 3rd Qu.:0.087868   3rd Qu.:5.268e-01   3rd Qu.: 0.72579   3rd Qu.:269.54
 Max.   :0.172340   Max.   :1.044e+03   Max.   :32.31526   Max.   :344.91
 kurtosis_volume
 Min.   :   150.5
 1st Qu.:  5622.3
 Median : 12158.7
 Mean   : 39749.6
 3rd Qu.: 72732.4
 Max.   :132136.7
```

```r
#| label: predictor-setup
#| message: false

library(ggplot2)

# Check that all test predictor levels occur in the training data
stopifnot(
  all(test$Re %in% train$Re),
  all(test$Fr %in% train$Fr),
  all(test$St %in% train$St)
)

# Use identical factor levels in training and test data
Re_levels <- sort(unique(train$Re))
Fr_levels <- sort(unique(train$Fr))
St_levels <- sort(unique(train$St))

train$Re_factor <- factor(train$Re, levels = Re_levels)
test$Re_factor  <- factor(test$Re, levels = Re_levels)

train$Fr_factor <- factor(train$Fr, levels = Fr_levels)
test$Fr_factor  <- factor(test$Fr, levels = Fr_levels)

# Inverse Froude number: Inf becomes 0
train$inv_Fr <- 1 / train$Fr
test$inv_Fr  <- 1 / test$Fr

# Compact summary of the original predictors
predictor_summary <- data.frame(
  Predictor = c("Re", "Fr", "St"),
```

```
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

# Explicitly match the displayed predictor order
predictor_summary <- data.frame(
  Predictor = predictors,
  Training_levels = sapply(train[predictors], function(x) length(unique(x))),
  Test_levels = sapply(test[predictors], function(x) length(unique(x))),
  Training_min = sapply(train[predictors], min),
  Training_max = sapply(train[predictors], max),
  row.names = NULL
)

knitr::kable(
  predictor_summary,
  caption = "Predictor ranges and number of distinct settings"
)
```

Table 3: Predictor ranges and number of distinct settings

Predictor	Training_levels	Test_levels	Training_min	Training_max
St	14	13	0.050	3
Re	3	3	90.000	398
Fr	3	3	0.052	Inf

```
knitr::kable(
  data.frame(Fr = Fr_levels, Inverse_Fr = 1 / Fr_levels),
  digits = 4,
  caption = "Froude number and its reciprocal"
)
```

Table 4: Froude number and its reciprocal

Fr	Inverse_Fr
0.052	19.2308
0.300	3.3333

Fr	Inverse_Fr
Inf	0.0000

```
#| label: predictor-counts
#| fig-width: 7
#| fig-height: 3.5

# Combine predictors only; no response values are used here
design_data <- rbind(
  data.frame(train[predictors], Dataset = "Training"),
  data.frame(test[predictors], Dataset = "Test")
)

design_data$Dataset <- factor(
  design_data$Dataset,
  levels = c("Training", "Test")
)

# Helper function for count plots
plot_setting_counts <- function(variable, setting_levels, axis_label) {

  plot_data <- data.frame(
    Setting = factor(design_data[[variable]], levels = setting_levels),
    Dataset = design_data$Dataset
  )

  ggplot(plot_data, aes(x = Setting, fill = Dataset)) +
    geom_bar(position = "dodge") +
    scale_fill_manual(values = c("Training" = "#2166AC", "Test" = "#D6604D")) +
    labs(
      x = axis_label,
      y = "Number of simulations",
      fill = "Dataset"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")
}

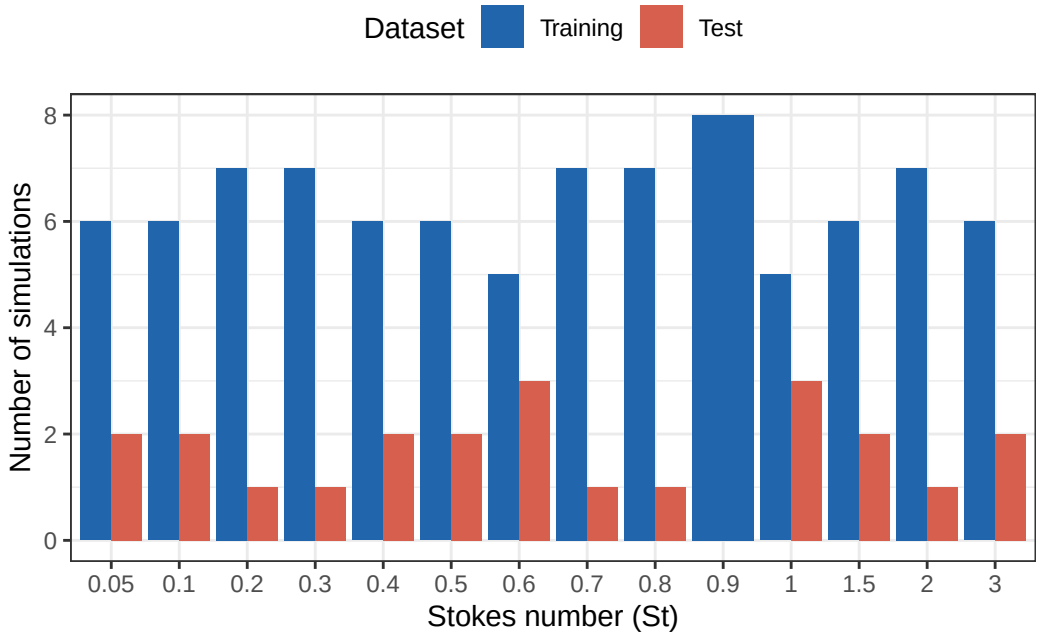
plot_setting_counts("Re", Re_levels, "Reynolds number (Re)")
```




```
plot_setting_counts("Fr", Fr_levels, "Froude number (Fr)")
```



```
plot_setting_counts("St", St_levels, "Stokes number (St)")
```




```
#| label: simulation-counts

# Include every combination, even combinations with no observations
design_counts <- expand.grid(
  Re = Re_levels,
  Fr = Fr_levels
)

design_counts <- design_counts[
  order(design_counts$Re, design_counts$Fr),
]

design_counts$Training <- mapply(
  function(re, fr) sum(train$Re == re & train$Fr == fr),
  design_counts$Re,
  design_counts$Fr
)

design_counts$Test <- mapply(
  function(re, fr) sum(test$Re == re & test$Fr == fr),
```

```
  design_counts$Re,
  design_counts$Fr
)

design_counts$Total <- design_counts$Training + design_counts$Test

knitr::kable(
  design_counts,
  row.names = FALSE,
  caption = "Simulation counts within each Re and Fr combination"
)
```

Table 5: Simulation counts within each Re and Fr combination

Re	Fr	Training	Test	Total
90	0.052	12	2	14
90	0.300	10	4	14
90	Inf	9	5	14
224	0.052	14	0	14
224	0.300	13	1	14
224	Inf	12	2	14
398	0.052	10	4	14
398	0.300	0	0	0
398	Inf	9	5	14

```
# Check for exact input settings shared by training and test data
shared_settings <- merge(
  unique(train[predictors]),
  unique(test[predictors]),
  by = predictors
)

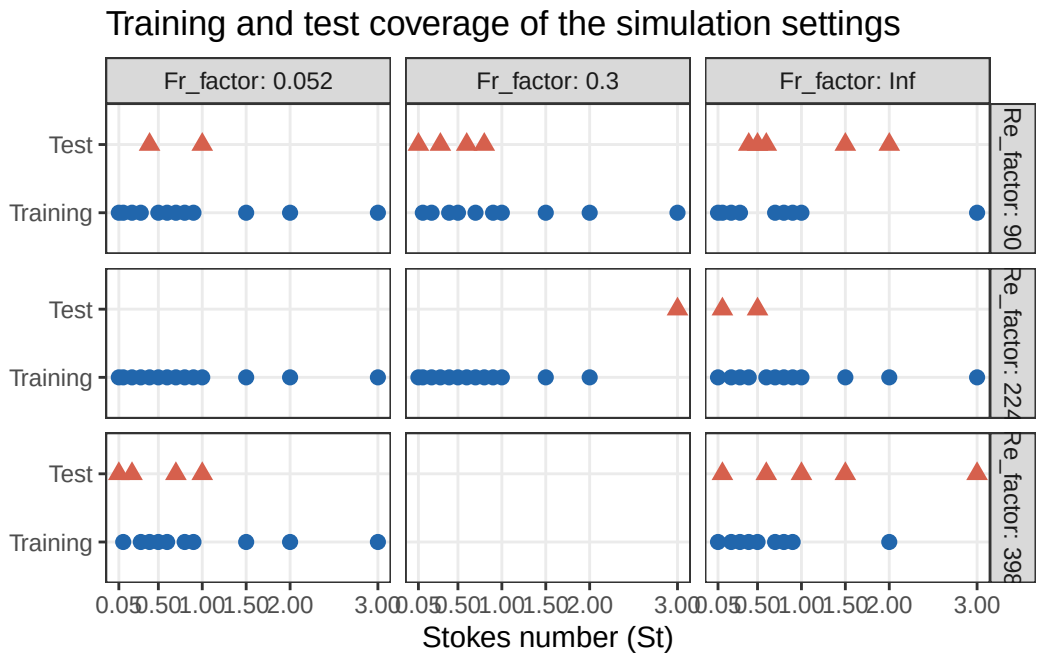
cat(
  "\nNumber of exact input settings shared by training and test:",
  nrow(shared_settings),
  "\n"
)
```

```
Number of exact input settings shared by training and test: 0
```

```r
#| label: simulation-coverage
#| fig-width: 8
#| fig-height: 6

design_data$Re_factor <- factor(design_data$Re, levels = Re_levels)
design_data$Fr_factor <- factor(design_data$Fr, levels = Fr_levels)

ggplot(
  design_data,
  aes(x = St, y = Dataset, color = Dataset, shape = Dataset)
) +
  geom_point(size = 2.5) +
  facet_grid(
    Re_factor ~ Fr_factor,
    labeller = label_both,
    drop = FALSE
  ) +
  scale_color_manual(
    values = c("Training" = "#2166AC", "Test" = "#D6604D")
  ) +
  scale_shape_manual(
    values = c("Training" = 16, "Test" = 17)
  ) +
  scale_x_continuous(
    breaks = c(0.05, 0.5, 1, 1.5, 2, 3)
  ) +
  labs(
    title = "Training and test coverage of the simulation settings",
    x = "Stokes number (St)",
    y = NULL
  ) +
  theme_bw(base_size = 11) +
  theme(
    legend.position = "none",
    panel.grid.minor = element_blank()
  )
```




```
#| label: response-plotting-data

response_columns <- c(
  "mean_volume",
  "sd_volume",
  "skewness_volume",
  "kurtosis_volume"
)

response_labels <- c(
  "Mean",
  "Standard deviation",
  "Skewness",
  "Kurtosis"
)

# Confirm that the responses exist and are finite
stopifnot(
  all(response_columns %in% names(train)),
  all(is.finite(as.matrix(train[response_columns]))))
)

# One row per simulation and response
```

```r
response_long <- do.call(
  rbind,
  lapply(seq_along(response_columns), function(j) {
    data.frame(
      St = train$St,
      Re_factor = train$Re_factor,
      Fr_factor = train$Fr_factor,
      Response = response_labels[j],
      Value = train[[response_columns[j]]]
    )
  })
)

response_long$Response <- factor(
  response_long$Response,
  levels = response_labels
)

# The log-scale plots below require positive values.
# Stop rather than silently removing nonpositive observations.
stopifnot(all(response_long$Value > 0))

# Consistent colors for Reynolds-number settings
Re_colors <- c(
  "90" = "#2166AC",
  "224" = "#1B7837",
  "398" = "#B2182B"
)
```

```r
#| label: response-distributions
#| fig-width: 8
#| fig-height: 5.5

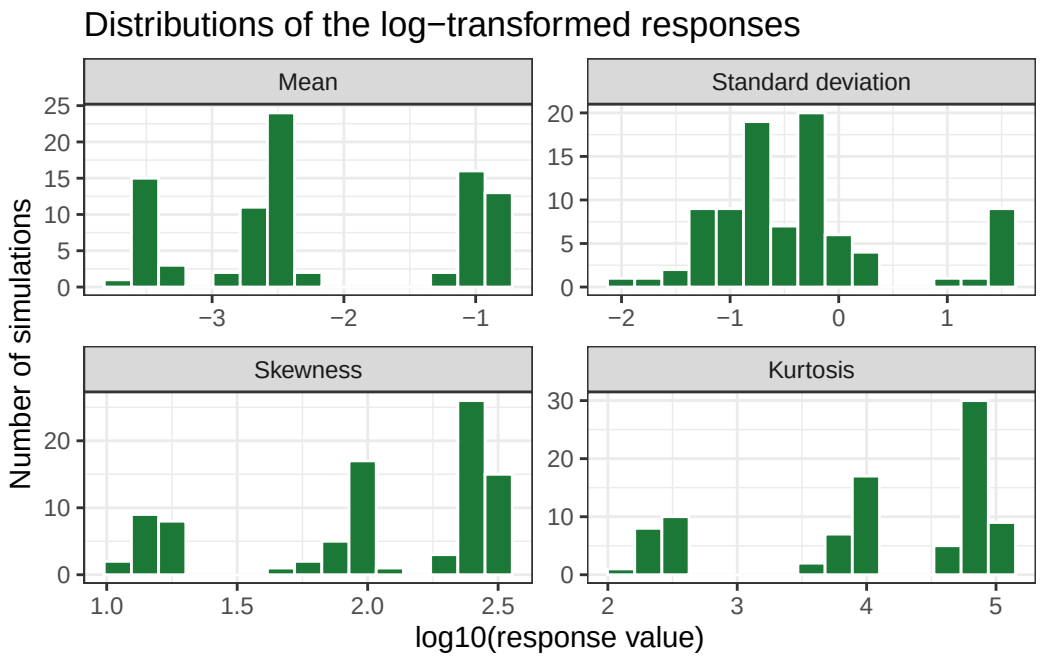
# Original-scale distributions
ggplot(response_long, aes(x = Value)) +
  geom_histogram(
    bins = 15,
    fill = "#2166AC",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
```

```
    scales = "free",
    ncol = 2
  ) +
  labs(
    title = "Distributions of the training responses",
    x = "Response value",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
# Distributions after taking log10
ggplot(response_long, aes(x = log10(Value))) +
  geom_histogram(
    bins = 15,
    fill = "#1B7837",
    color = "white"
  ) +
  facet_wrap(
    ~ Response,
    scales = "free",
    ncol = 2
  ) +
```

```
  labs(
    title = "Distributions of the log-transformed responses",
    x = "log10(response value)",
    y = "Number of simulations"
  ) +
  theme_bw(base_size = 11)
```




```
#| label: responses-versus-St
#| fig-width: 8
#| fig-height: 3.8

for (response_name in response_labels) {

  plot_data <- response_long[
    response_long$Response == response_name,
  ]

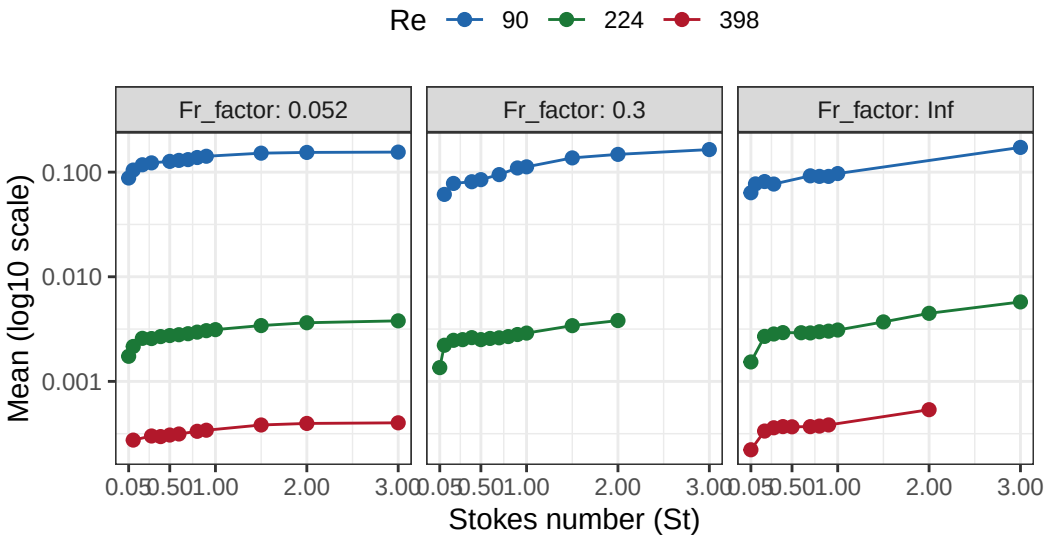
  p <- ggplot(
    plot_data,
    aes(
      x = St,
      y = Value,
```

```
      color = Re_factor,
      group = Re_factor
    )
  ) +
    geom_line(linewidth = 0.5) +
    geom_point(size = 2) +
    facet_wrap(
      ~ Fr_factor,
      nrow = 1,
      labeller = label_both,
      drop = FALSE
    ) +
    scale_color_manual(values = Re_colors) +
    scale_y_log10() +
    scale_x_continuous(
      breaks = c(0.05, 0.5, 1, 2, 3)
    ) +
    labs(
      title = paste(response_name, "versus Stokes number"),
      subtitle = "Lines connect available training observations within each Re-Fr regime",
      x = "Stokes number (St)",
      y = paste(response_name, "(log10 scale)"),
      color = "Re"
    ) +
    theme_bw(base_size = 11) +
    theme(legend.position = "top")

  print(p)
}
```

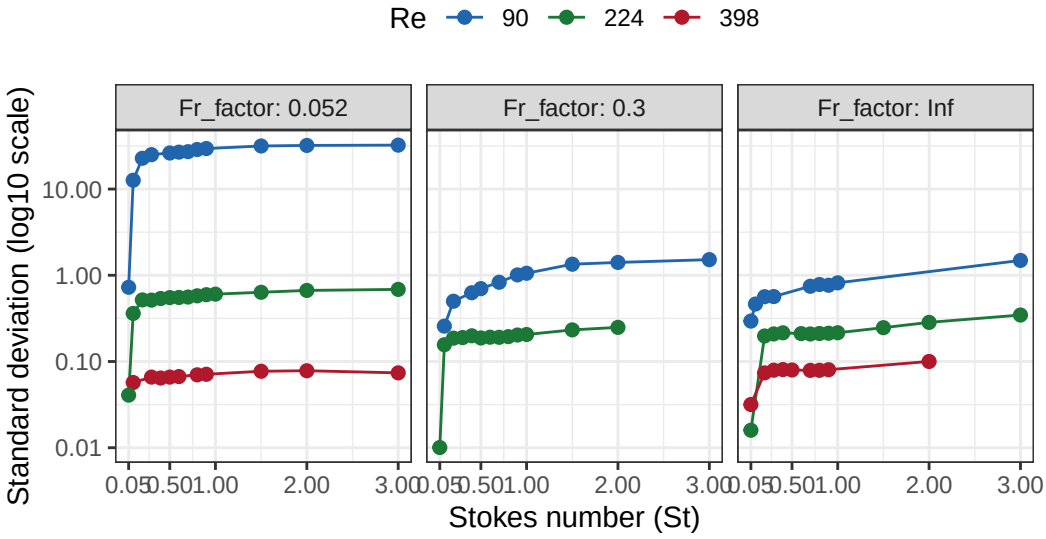
Mean versus Stokes number

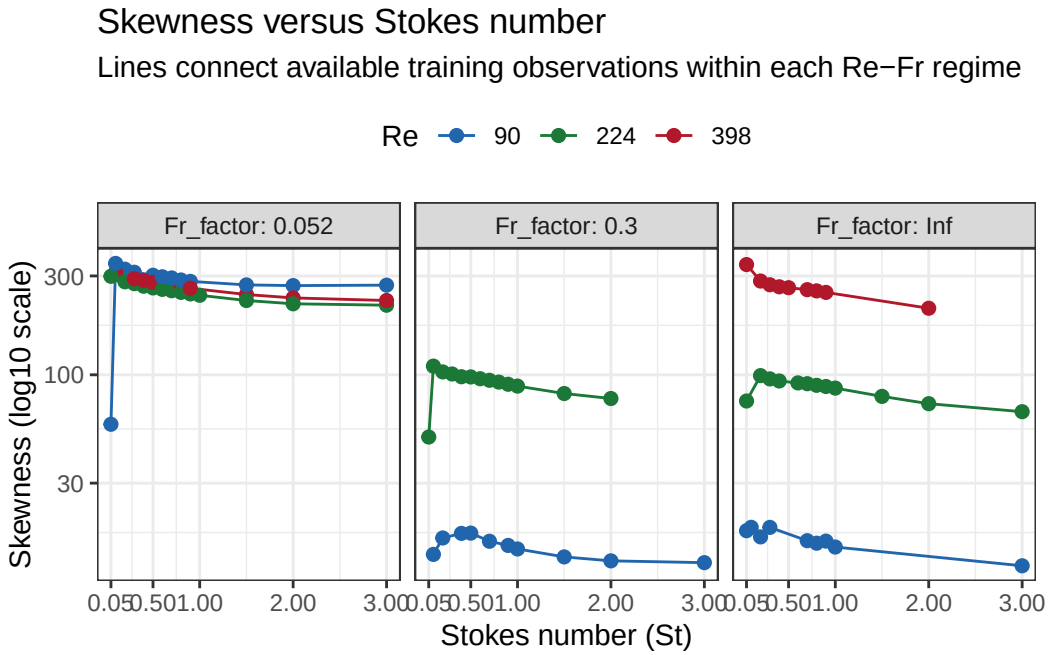
Lines connect available training observations within each Re−Fr regime

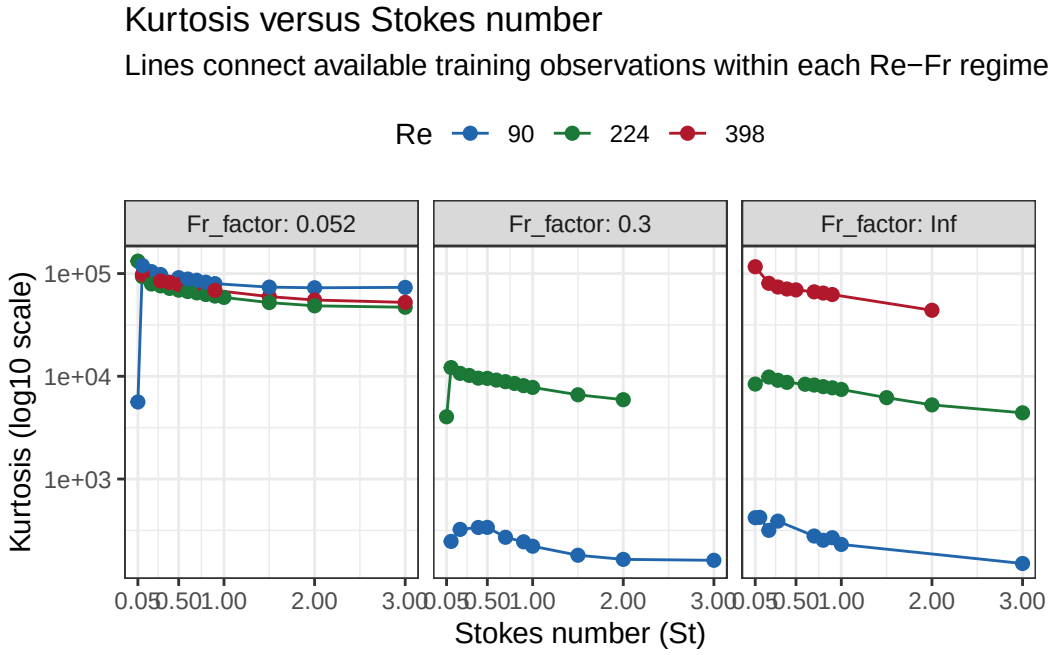


Standard deviation versus Stokes number

Lines connect available training observations within each Re−Fr regime

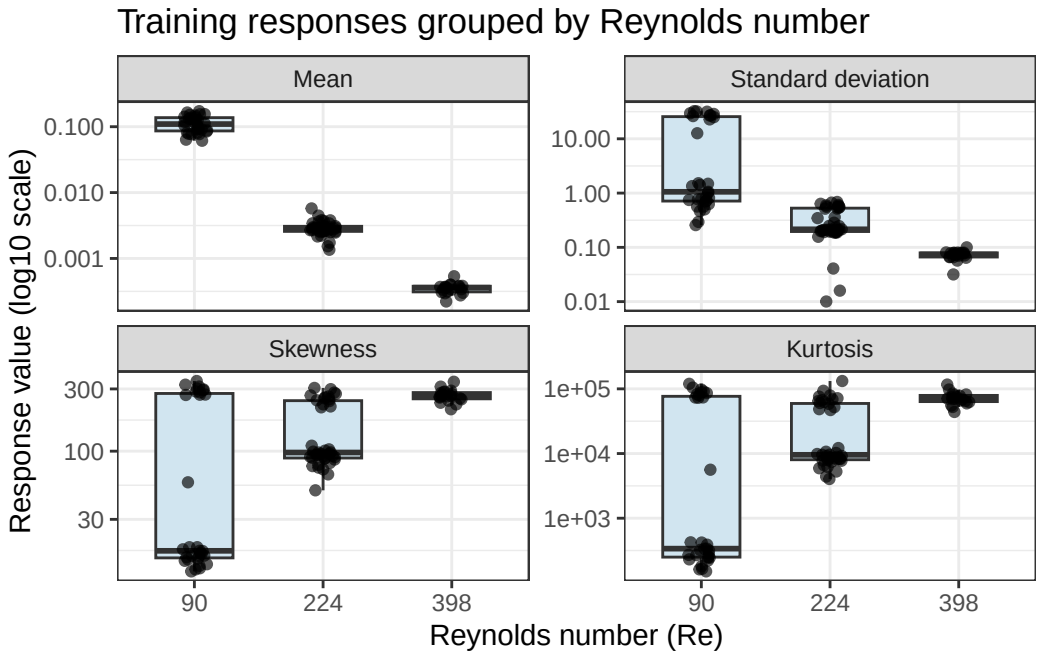






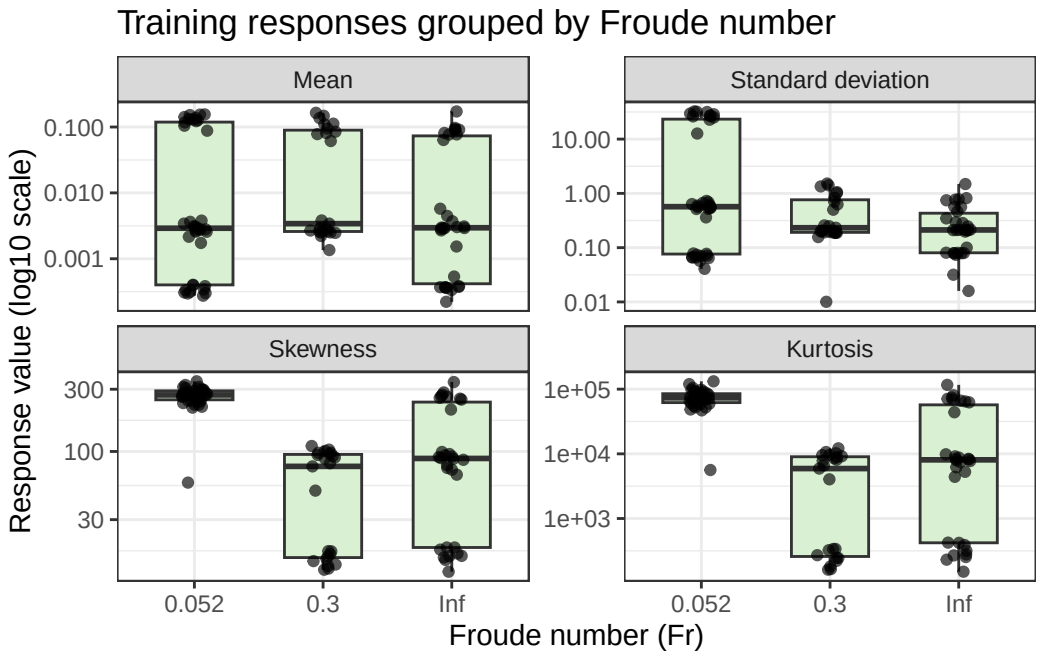

```
#| label: responses-by-Re-and-Fr
#| fig-width: 8
#| fig-height: 6
```

```r
# Responses grouped by Reynolds number
ggplot(
  response_long,
  aes(x = Re_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D1E5F0",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Reynolds number",
    x = "Reynolds number (Re)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```




```
# Responses grouped by Froude number
ggplot(
  response_long,
  aes(x = Fr_factor, y = Value)
) +
  geom_boxplot(
    fill = "#D9F0D3",
    outlier.shape = NA,
    width = 0.6
  ) +
  geom_point(
    position = position_jitter(
      width = 0.10,
      height = 0,
      seed = 325
    ),
    alpha = 0.65,
    size = 1.5
  ) +
  scale_y_log10() +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Training responses grouped by Froude number",
```

```
    x = "Froude number (Fr)",
    y = "Response value (log10 scale)"
  ) +
  theme_bw(base_size = 11)
```




```
#| label: candidate-transformations

# Check domains before taking logarithms
stopifnot(
  all(train$St > 0), all(test$St > 0),
  all(train$Re > 0), all(test$Re > 0),
  all(as.matrix(train[response_columns]) > 0)
)

# Candidate predictor transformations
train$log10_St <- log10(train$St)
test$log10_St  <- log10(test$St)

train$log10_Re <- log10(train$Re)
test$log10_Re  <- log10(test$Re)

# Candidate response transformations: training data only
```

```r
for (response in response_columns) {
  train[[paste0("log10_", response)]] <- log10(train[[response]])
}

# Predictor transformation reference
transformation_reference <- data.frame(
  Predictor = c("St", "Re", "Fr"),
  Candidate = c("log10(St)", "log10(Re)", "1/Fr"),
  Purpose = c(
    "Represent proportional changes in particle inertia",
    "Represent proportional changes in Reynolds number",
    "Represent relative gravity strength; Inf becomes zero"
  )
)

knitr::kable(
  transformation_reference,
  caption = "Candidate predictor transformations"
)
```

Table 6: Candidate predictor transformations

Predictor	Candidate	Purpose
St	log10(St)	Represent proportional changes in particle inertia
Re	log10(Re)	Represent proportional changes in Reynolds number
Fr	1/Fr	Represent relative gravity strength; Inf becomes zero

```r
#| label: small-St-investigation
#| fig-width: 8
#| fig-height: 8

small_St <- train[train$St <= 0.3, ]

small_St <- small_St[
  order(small_St$Re, small_St$Fr, small_St$St),
]

small_St_table <- small_St[
  c(
    "Re", "Fr", "St",
    "sd_volume", "skewness_volume", "kurtosis_volume"
```

```
  )
]

# Round only the displayed copy
small_St_table[] <- lapply(
  small_St_table,
  function(x) signif(x, 4)
)

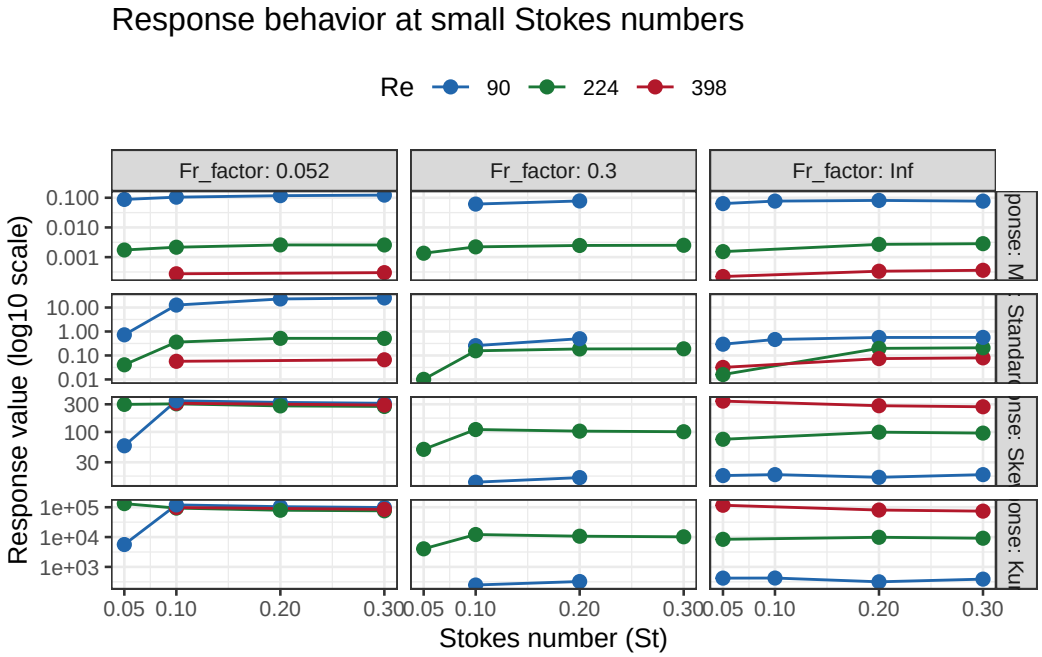
knitr::kable(
  small_St_table,
  row.names = FALSE,
  caption = "Response summaries at small Stokes numbers"
)
```

Table 7: Response summaries at small Stokes numbers

Re	Fr	St	sd_volume	skewness_volume	kurtosis_volume
90	0.052	0.05	0.72580	57.71	5622.0
90	0.052	0.10	12.65000	344.90	119700.0
90	0.052	0.20	22.75000	323.40	104500.0
90	0.052	0.30	25.05000	312.50	97850.0
90	0.300	0.10	0.25710	13.59	247.0
90	0.300	0.20	0.49990	16.31	323.5
90	Inf	0.05	0.29430	17.68	420.3
90	Inf	0.10	0.46390	18.36	423.4
90	Inf	0.20	0.56380	16.50	316.7
90	Inf	0.30	0.56620	18.35	389.2
224	0.052	0.05	0.04075	298.90	132100.0
224	0.052	0.10	0.36100	305.40	93370.0
224	0.052	0.20	0.51800	281.20	79120.0
224	0.052	0.30	0.51520	275.40	75840.0
224	0.300	0.05	0.01006	50.08	4035.0
224	0.300	0.10	0.15640	110.10	12160.0
224	0.300	0.20	0.18660	103.20	10670.0
224	0.300	0.30	0.18940	100.90	10200.0
224	Inf	0.05	0.01594	74.75	8402.0
224	Inf	0.20	0.19750	99.10	9838.0
224	Inf	0.30	0.20880	95.54	9142.0
398	0.052	0.10	0.05705	312.40	97640.0
398	0.052	0.30	0.06594	291.00	84720.0

Re	Fr	St	sd_volume	skewness_volume	kurtosis_volume
398	Inf	0.05	0.03171	340.50	116500.0
398	Inf	0.20	0.07383	283.50	80390.0
398	Inf	0.30	0.07926	272.00	73990.0

```
ggplot(
  response_long[response_long$St <= 0.3, ],
  aes(
    x = St,
    y = Value,
    color = Re_factor,
    group = Re_factor
  )
) +
  geom_line(linewidth = 0.5) +
  geom_point(size = 2) +
  scale_color_manual(values = Re_colors) +
  scale_y_log10() +
  scale_x_continuous(breaks = c(0.05, 0.1, 0.2, 0.3)) +
  facet_grid(
    Response ~ Fr_factor,
    scales = "free_y",
    labeller = label_both
  ) +
  labs(
    title = "Response behavior at small Stokes numbers",
    x = "Stokes number (St)",
    y = "Response value (log10 scale)",
    color = "Re"
  ) +
  theme_bw(base_size = 10) +
  theme(legend.position = "top")
```




```
#| label: compare-St-transformations
#| fig-width: 9
#| fig-height: 5.5

# Two representations of the same observations
St_scale_data <- rbind(
  data.frame(
    response_long,
    X = response_long$St,
    Representation = "St"
  ),
  data.frame(
    response_long,
    X = log10(response_long$St),
    Representation = "log10(St)"
  )
)

St_scale_data$Representation <- factor(
  St_scale_data$Representation,
  levels = c("St", "log10(St)")
)
```

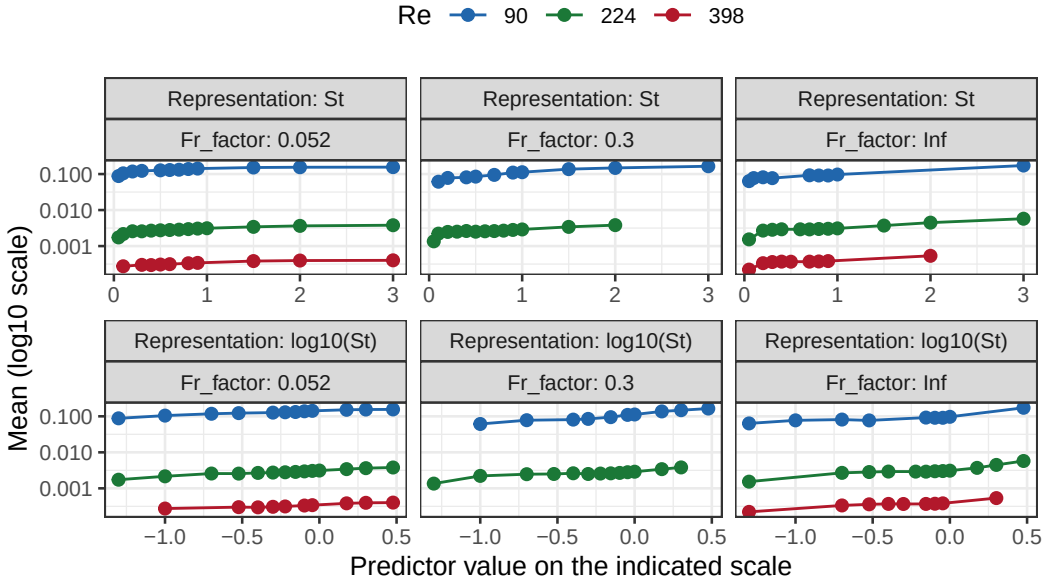
```r
for (response_name in response_labels) {

  plot_data <- St_scale_data[
    St_scale_data$Response == response_name,
  ]

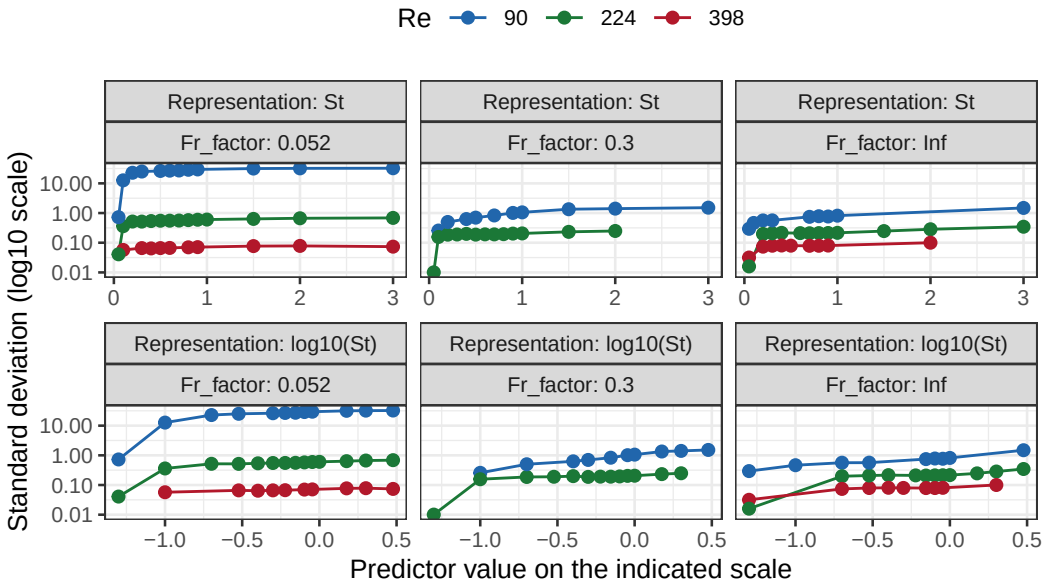
  p <- ggplot(
    plot_data,
    aes(
      x = X,
      y = Value,
      color = Re_factor,
      group = Re_factor
    )
  ) +
    geom_line(linewidth = 0.5) +
    geom_point(size = 1.8) +
    scale_color_manual(values = Re_colors) +
    scale_y_log10() +
    facet_wrap(
      vars(Representation, Fr_factor),
      ncol = 3,
      scales = "free_x",
      labeller = label_both
    ) +
    labs(
      title = paste("Comparing St representations:", response_name),
      x = "Predictor value on the indicated scale",
      y = paste(response_name, "(log10 scale)"),
      color = "Re"
    ) +
    theme_bw(base_size = 10) +
    theme(legend.position = "top")

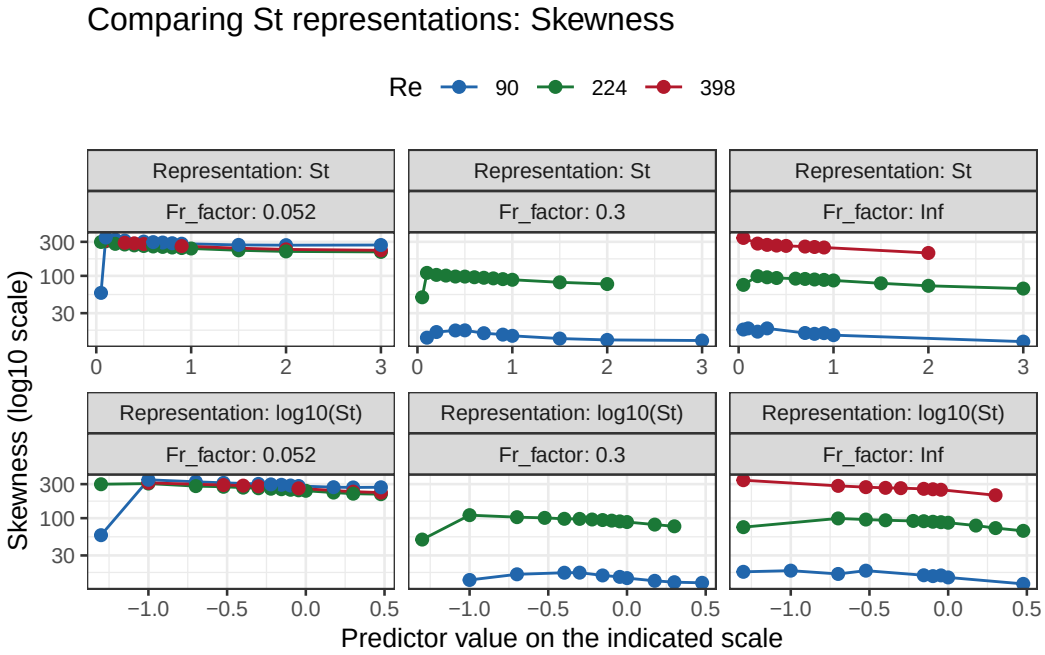
  print(p)
}
```

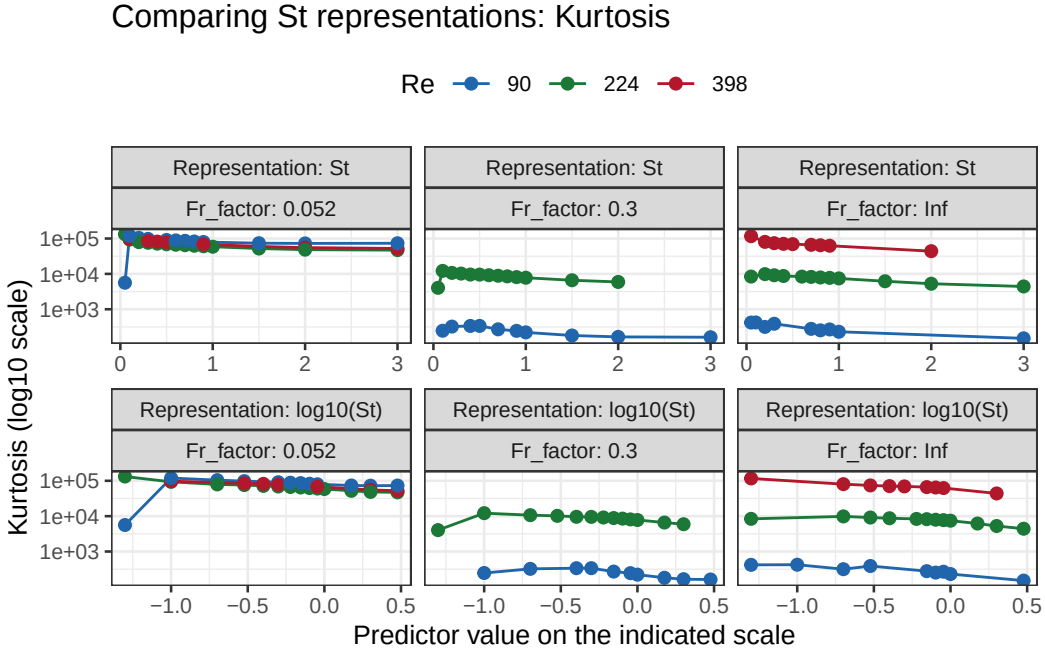
Comparing St representations: Mean



Comparing St representations: Standard deviation








```
#| label: matched-gravity-contrasts
#| fig-width: 8
#| fig-height: 6
```

```r
matching_columns <- c("St", "Re_factor", "Response", "Value")

strong_gravity <- response_long[
  response_long$Fr_factor == "0.052",
  matching_columns
]

no_gravity <- response_long[
  response_long$Fr_factor == "Inf",
  matching_columns
]

# Match on all remaining predictors and the response being compared
gravity_contrasts <- merge(
  strong_gravity,
  no_gravity,
  by = c("St", "Re_factor", "Response"),
  suffixes = c("_strong", "_none")
)

gravity_contrasts$log10_ratio <- log10(
  gravity_contrasts$Value_strong /
    gravity_contrasts$Value_none
)

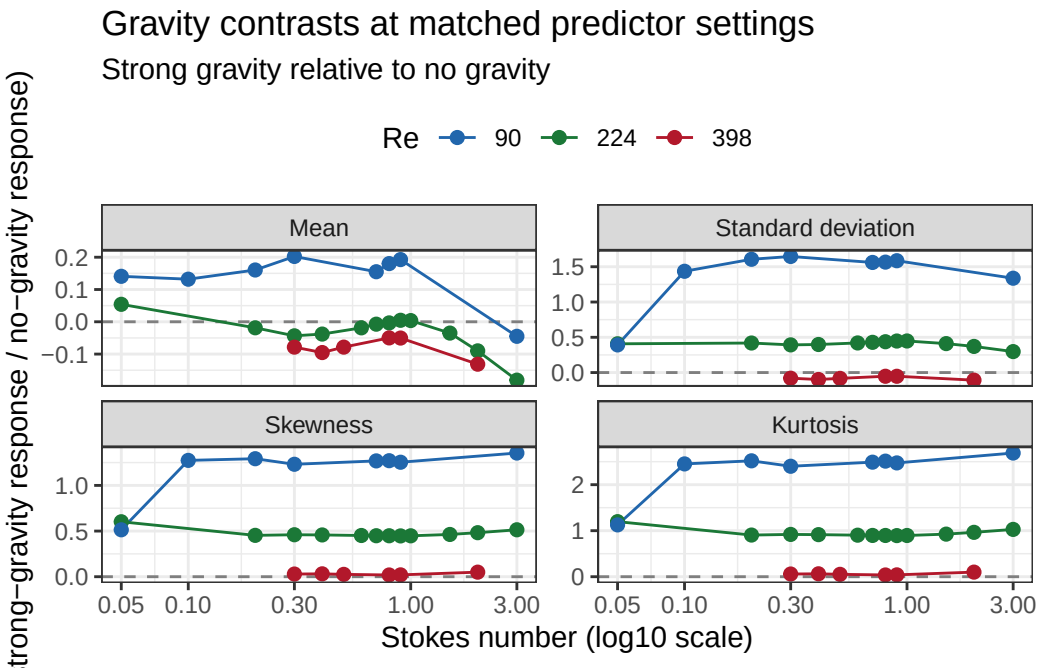
# Show how many matched settings support each Re comparison
matched_counts <- unique(
  gravity_contrasts[c("Re_factor", "St")]
)

knitr::kable(
  as.data.frame(table(Re = matched_counts$Re_factor)),
  caption = "Number of matched St settings for the gravity comparison"
)
```

Table 8: Number of matched St settings for the gravity comparison

Re	Freq
90	8
224	12
398	6

```r
ggplot(
  gravity_contrasts,
  aes(
    x = St,
    y = log10_ratio,
    color = Re_factor,
    group = Re_factor
  )
) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed",
    color = "grey50"
  ) +
  geom_line(linewidth = 0.5) +
  geom_point(size = 2) +
  scale_color_manual(values = Re_colors) +
  scale_x_log10(
    breaks = c(0.05, 0.1, 0.3, 1, 3)
  ) +
  facet_wrap(~ Response, scales = "free_y", ncol = 2) +
  labs(
    title = "Gravity contrasts at matched predictor settings",
    subtitle = "Strong gravity relative to no gravity",
    x = "Stokes number (log10 scale)",
    y = "log10(strong-gravity response / no-gravity response)",
    color = "Re"
  ) +
  theme_bw(base_size = 11) +
  theme(legend.position = "top")
```




```
#| label: create-validation-folds

# Reproducible random assignment
set.seed(325)

K <- 5L

# Stable identifiers matching the current training-data row order
train$row_id <- seq_len(nrow(train))

# Identify the observed simulation regimes
train$regime <- interaction(
  train$Re_factor,
  train$Fr_factor,
  drop = TRUE,
  sep = " / "
)

train$cv_fold <- NA_integer_

# Randomize the labels of the five folds
fold_order <- sample(seq_len(K))
```

```r
# Cycle through fold labels across regimes.
# This keeps total fold sizes and within-regime counts balanced.
offset <- 0L

for (regime_name in levels(train$regime)) {

  rows <- which(train$regime == regime_name)

  assignments <- fold_order[
    ((seq_along(rows) - 1L + offset) %% K) + 1L
  ]

  # Randomly assign these fold labels to observations in this regime
  train$cv_fold[rows] <- assignments[
    sample.int(length(rows))
  ]

  offset <- offset + length(rows)
}

# Save row indices for each training/validation split
cv_folds <- lapply(seq_len(K), function(k) {
  list(
    analysis = which(train$cv_fold != k),
    assessment = which(train$cv_fold == k)
  )
})

names(cv_folds) <- paste0("Fold_", seq_len(K))
```

```r
#| label: check-validation-folds

# Every row must have a valid fold assignment
stopifnot(
  !anyNA(train$cv_fold),
  all(train$cv_fold %in% seq_len(K))
)

# Every observation must appear in exactly one assessment set
assessment_rows <- unlist(
  lapply(cv_folds, function(x) x$assessment),
  use.names = FALSE
```

```
)

stopifnot(
  identical(sort(assessment_rows), seq_len(nrow(train)))
)

# Check separation and regime coverage in every split
for (k in seq_len(K)) {

  analysis_rows <- cv_folds[[k]]$analysis
  assessment_rows <- cv_folds[[k]]$assessment

  stopifnot(
    length(intersect(analysis_rows, assessment_rows)) == 0L,
    length(analysis_rows) + length(assessment_rows) == nrow(train),
    all(
      levels(train$regime) %in%
        as.character(train$regime[analysis_rows])
    ),
    all(
      levels(train$regime) %in%
        as.character(train$regime[assessment_rows])
    )
  )
}

# Overall fold sizes
fold_summary <- data.frame(
  Fold = seq_len(K),
  Training_rows = vapply(
    cv_folds,
    function(x) length(x$analysis),
    integer(1)
  ),
  Validation_rows = vapply(
    cv_folds,
    function(x) length(x$assessment),
    integer(1)
  )
)

knitr::kable(
```

```
  fold_summary,
  row.names = FALSE,
  caption = "Training and validation sample sizes"
)
```

Table 9: Training and validation sample sizes

Fold	Training_rows	Validation_rows
1	71	18
2	72	17
3	71	18
4	71	18
5	71	18

```
# Counts of held-out observations by regime and fold
regime_fold_counts <- table(
  Regime = train$regime,
  Fold = factor(train$cv_fold, levels = seq_len(K))
)

regime_fold_table <- data.frame(
  Regime = rownames(regime_fold_counts),
  as.data.frame.matrix(regime_fold_counts),
  row.names = NULL,
  check.names = FALSE
)

names(regime_fold_table)[-1] <- paste0("Fold ", seq_len(K))

knitr::kable(
  regime_fold_table,
  caption = "Validation observations from each Re / Fr regime"
)
```

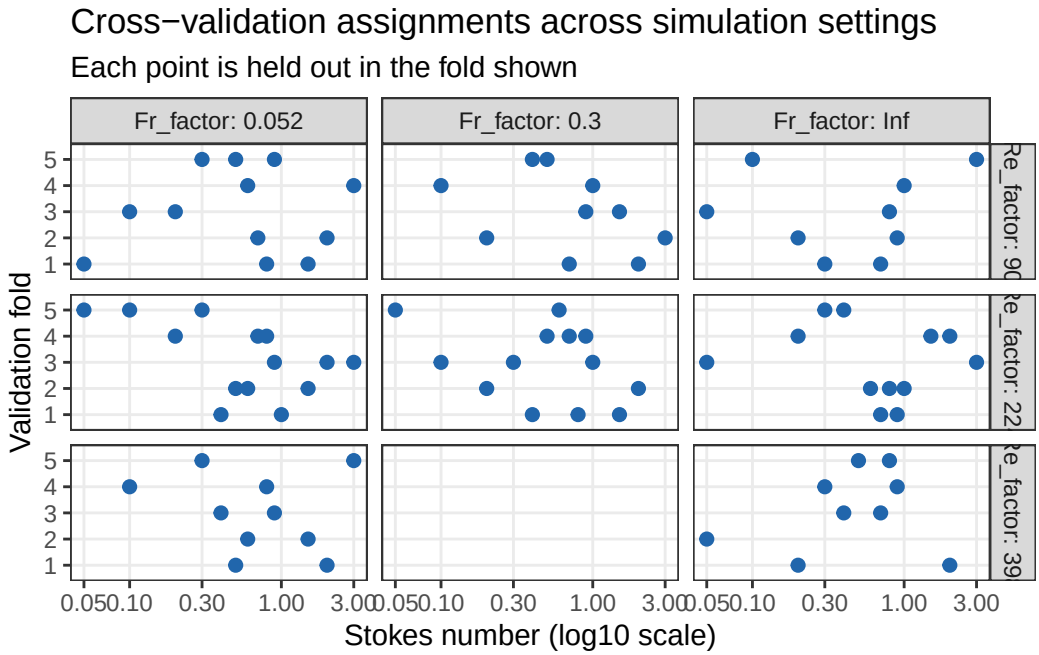
Table 10: Validation observations from each Re / Fr regime

Regime	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
90 / 0.052	3	2	2	2	3
224 / 0.052	2	3	3	3	3
398 / 0.052	2	2	2	2	2

Regime	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
90 / 0.3	2	2	2	2	2
224 / 0.3	3	2	3	3	2
90 / Inf	2	2	2	1	2
224 / Inf	2	3	2	3	2
398 / Inf	2	1	2	2	2

```
#| label: validation-design-plot
#| fig-width: 8
#| fig-height: 6

ggplot(
  train,
  aes(
    x = St,
    y = factor(cv_fold, levels = seq_len(K))
  )
) +
  geom_point(
    color = "#2166AC",
    size = 2
  ) +
  scale_x_log10(
    breaks = c(0.05, 0.1, 0.3, 1, 3)
  ) +
  facet_grid(
    Re_factor ~ Fr_factor,
    labeller = label_both,
    drop = FALSE
  ) +
  labs(
    title = "Cross-validation assignments across simulation settings",
    subtitle = "Each point is held out in the fold shown",
    x = "Stokes number (log10 scale)",
    y = "Validation fold"
  ) +
  theme_bw(base_size = 11) +
  theme(panel.grid.minor = element_blank())
```




```
# Check the distribution of small-St observations
small_St_counts <- data.frame(
  Fold = seq_len(K),
  Small_St_validation_rows = vapply(
    cv_folds,
    function(x) {
      as.integer(sum(train$St[x$assessment] <= 0.3))
    },
    integer(1)
  )
)

knitr::kable(
  small_St_counts,
  caption = "Validation observations with St at or below 0.3"
)
```

Table 11: Validation observations with St at or below 0.3

	Fold	Small_St_validation_rows
Fold_1	1	3
Fold_2	2	4

	Fold	Small_St_validation_rows
Fold_3	3	6
Fold_4	4	5
Fold_5	5	8

```
#| label: validation-scoring-functions

# Responses to evaluate separately
cv_responses <- c(
  "mean_volume",
  "sd_volume",
  "skewness_volume",
  "kurtosis_volume"
)

# Score original-scale observed values and held-out predictions
score_predictions <- function(observed, predicted) {

  stopifnot(
    is.numeric(observed),
    is.numeric(predicted),
    length(observed) == length(predicted),
    length(observed) > 0L,
    all(is.finite(observed)),
    all(is.finite(predicted))
  )

  errors <- observed - predicted

  data.frame(
    RMSE = sqrt(mean(errors^2)),
    MAE = mean(abs(errors)),
    Nonpositive_predictions = sum(predicted <= 0)
  )
}

# Retain the validation settings for later modeling steps
cv_settings <- list(
  seed = 325L,
  folds = K,
  responses = cv_responses,
```

```
  primary_metric = "Original-scale RMSE",
  secondary_metric = "Original-scale MAE"
)

cat(
  "Validation setup complete:",
  K, "folds;",
  length(cv_responses), "responses;",
  "primary metric =", cv_settings$primary_metric,
  "\n"
)
```

```
Validation setup complete: 5 folds; 4 responses; primary metric = Original-scale RMSE
```

```
#| label: fit-baseline-models

# Match response labels to their column names
baseline_labels <- setNames(response_labels, response_columns)

# Explicit predictor list excludes row IDs and fold assignments
baseline_predictors <- c("Re", "Fr_factor", "St")

baseline_models <- setNames(
  lapply(cv_responses, function(response) {

    model_formula <- reformulate(
      baseline_predictors,
      response = response
    )

    fit <- lm(model_formula, data = train)

    # Check that all model coefficients are identifiable
    stopifnot(fit$rank == ncol(model.matrix(fit)))

    fit
  }),
  cv_responses
)

# Store detailed summaries for later inspection
baseline_summaries <- lapply(baseline_models, summary)
```

```r
# Compact description of training fit
baseline_training_summary <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    fit <- baseline_models[[response]]
    model_summary <- summary(fit)

    data.frame(
      Response = baseline_labels[[response]],
      R_squared = model_summary$r.squared,
      Adjusted_R_squared = model_summary$adj.r.squared,
      Training_RMSE = sqrt(mean(residuals(fit)^2))
    )
  })
)

knitr::kable(
  baseline_training_summary,
  digits = 4,
  row.names = FALSE,
  caption = "Main-effects linear models: training fit"
)
```

Table 12: Main-effects linear models: training fit

Response	R_squared	Adjusted_R_squared	Training_RMSE
Mean	0.6566	0.6403	0.0325
Standard deviation	0.4412	0.4146	6.6504
Skewness	0.7720	0.7612	53.6970
Kurtosis	0.7114	0.6977	20352.0226

```r
#| label: baseline-cross-validation

baseline_oof_parts <- list()
part_number <- 1L

for (response in cv_responses) {

  model_formula <- reformulate(
```

```
    baseline_predictors,
    response = response
  )

  for (k in seq_len(K)) {

    analysis_rows <- cv_folds[[k]]$analysis
    assessment_rows <- cv_folds[[k]]$assessment

    analysis_data <- train[analysis_rows, ]
    assessment_data <- train[assessment_rows, ]

    fold_fit <- lm(model_formula, data = analysis_data)

    stopifnot(
      fold_fit$rank == ncol(model.matrix(fold_fit))
    )

    fold_predictions <- as.numeric(
      predict(fold_fit, newdata = assessment_data)
    )

    baseline_oof_parts[[part_number]] <- data.frame(
      row_id = assessment_data$row_id,
      Response = response,
      Fold = k,
      Re = assessment_data$Re,
      Fr = assessment_data$Fr,
      St = assessment_data$St,
      Observed = assessment_data[[response]],
      Predicted = fold_predictions,
      Mean_only_prediction = mean(analysis_data[[response]])
    )

    part_number <- part_number + 1L
  }
}

# OOF means out-of-fold: every prediction comes from a fit
# that excluded the observation being predicted.
baseline_oof <- do.call(rbind, baseline_oof_parts)
```

```r
baseline_cv_summary <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    predictions <- baseline_oof[
      baseline_oof$Response == response,
    ]

    # Exactly one held-out prediction per training observation
    stopifnot(
      nrow(predictions) == nrow(train),
      identical(
        sort(predictions$row_id),
        sort(train$row_id)
      )
    )

    linear_scores <- score_predictions(
      predictions$Observed,
      predictions$Predicted
    )

    mean_scores <- score_predictions(
      predictions$Observed,
      predictions$Mean_only_prediction
    )

    data.frame(
      Response = baseline_labels[[response]],
      CV_RMSE = linear_scores$RMSE,
      CV_MAE = linear_scores$MAE,
      Mean_only_RMSE = mean_scores$RMSE,
      Nonpositive = linear_scores$Nonpositive_predictions
    )
  })
)

knitr::kable(
  baseline_cv_summary,
  digits = 4,
  row.names = FALSE,
  caption = "Original-scale out-of-fold prediction performance"
```

```
)
```

Table 13: Original-scale out-of-fold prediction performance

Response	CV_RMSE	CV_MAE	Mean_only_RMSE	Nonpositive
Mean	0.0344	0.0297	0.0557	19
Standard deviation	6.9111	5.2599	8.9166	23
Skewness	55.3672	43.8728	112.7479	1
Kurtosis	21207.6625	16246.7159	38081.1333	7

```
#| label: baseline-diagnostics
#| fig-width: 8
#| fig-height: 7

plot_baseline_diagnostics <- function(model, response_label) {

  old_par <- par(
    mfrow = c(2, 2),
    oma = c(0, 0, 3, 0),
    mar = c(4, 4, 2, 1)
  )

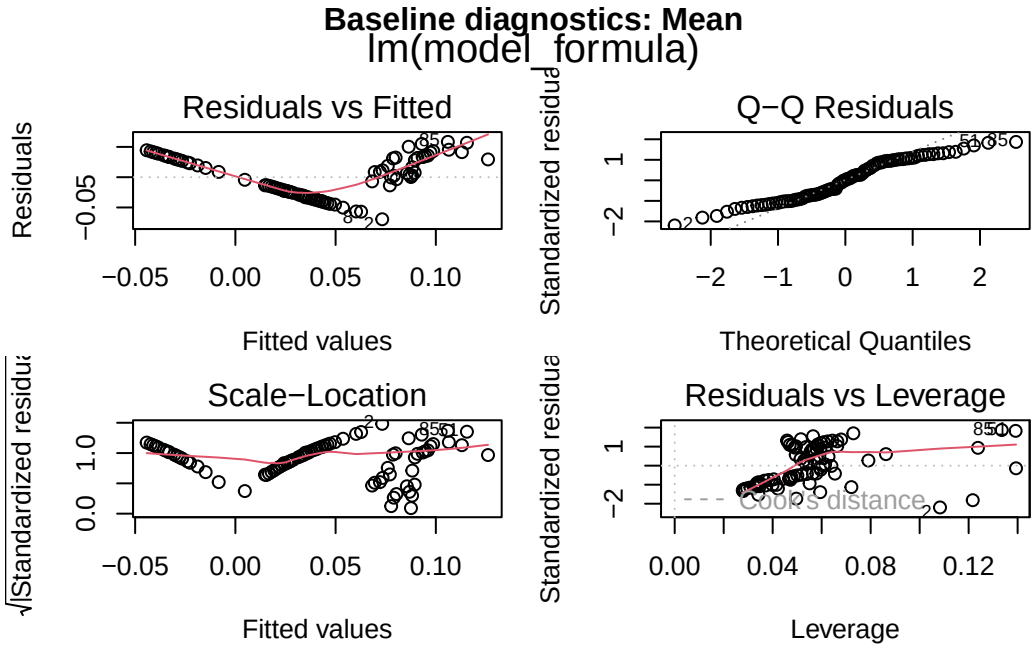
  on.exit(par(old_par))

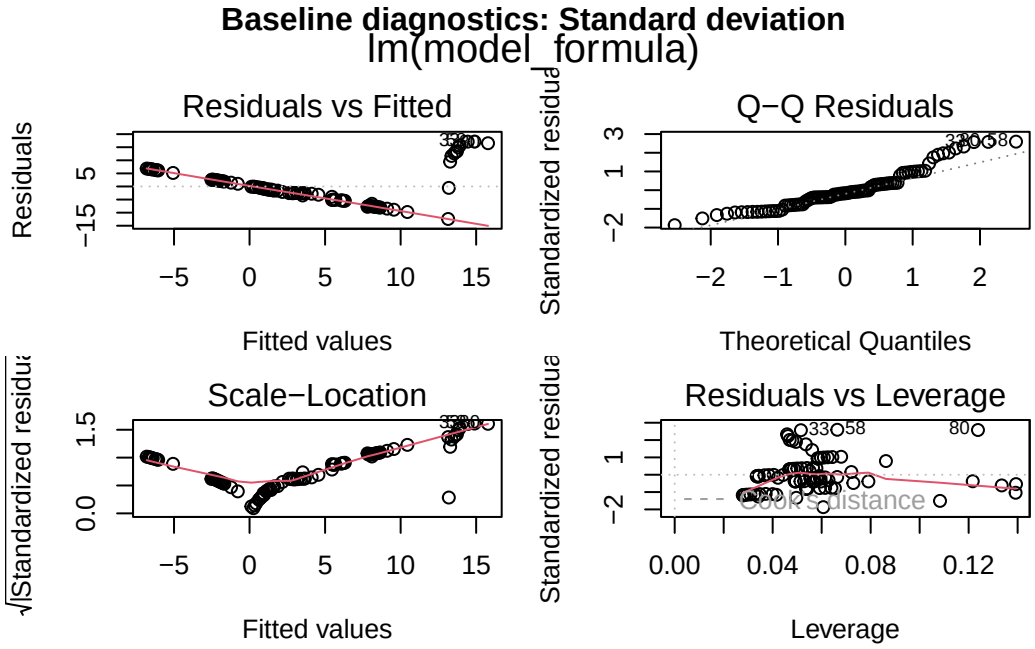
  plot(
    model,
    which = c(1, 2, 3, 5),
    ask = FALSE
  )

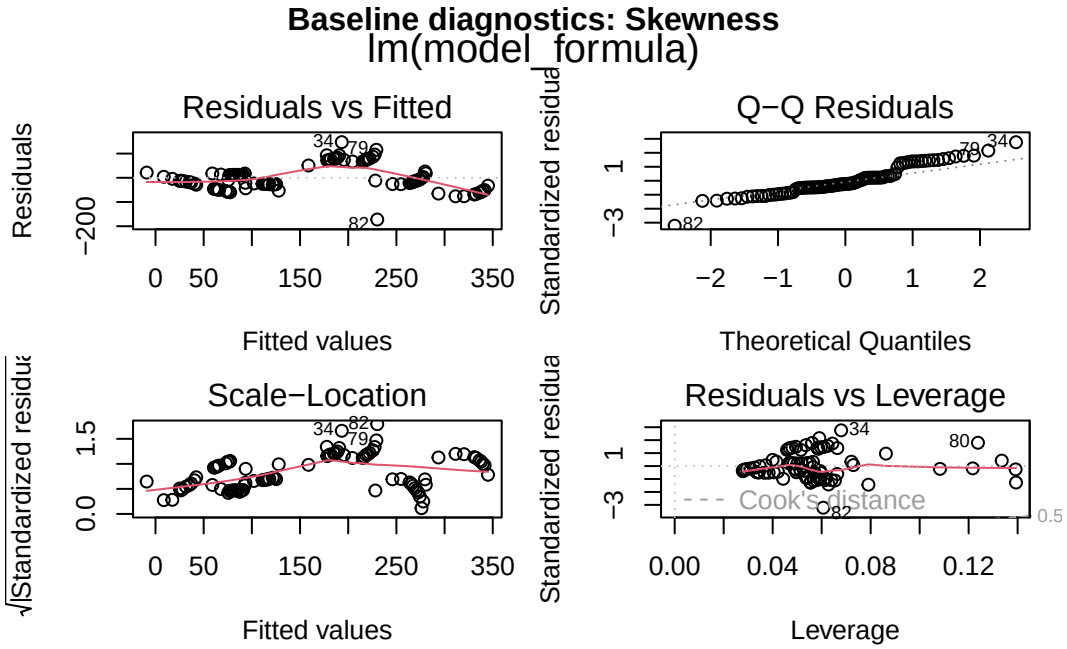
  mtext(
    paste("Baseline diagnostics:", response_label),
    outer = TRUE,
    line = 1,
    font = 2
  )
}

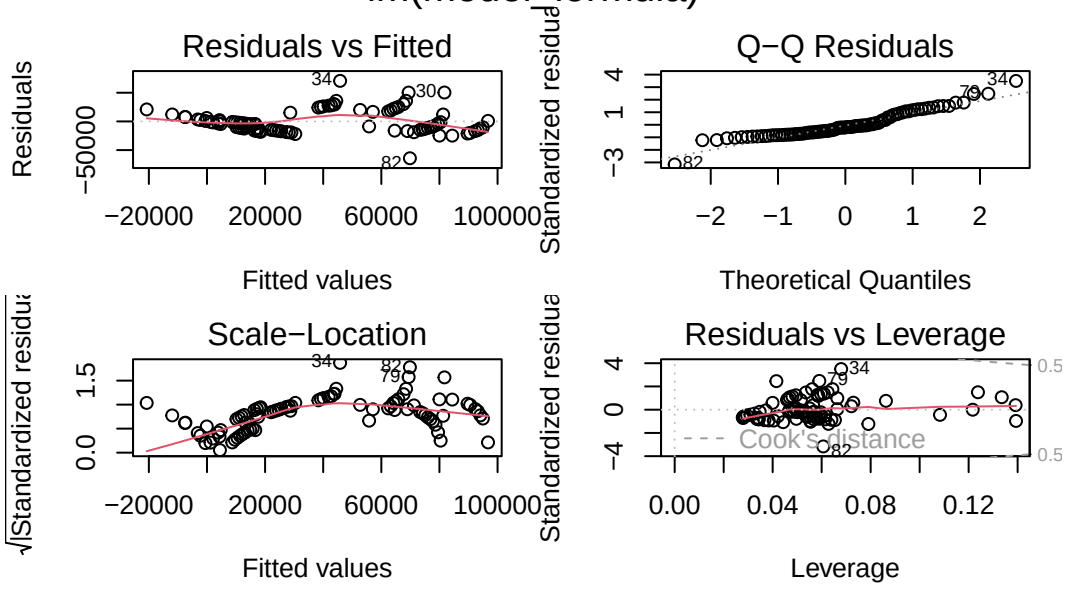
for (response in cv_responses) {
  plot_baseline_diagnostics(
    baseline_models[[response]],
```

```
    baseline_labels[[response]]
  )
}
```










```
#| label: baseline-error-patterns
#| fig-width: 8
#| fig-height: 8
```

```r
baseline_oof$Error <- (
  baseline_oof$Observed - baseline_oof$Predicted
)

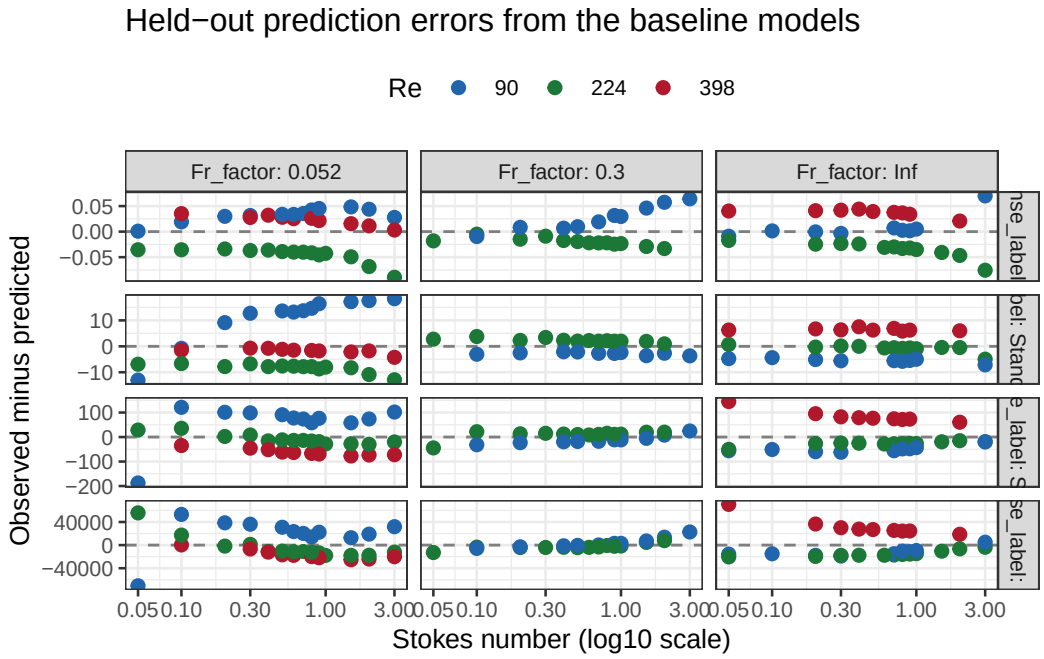
baseline_oof$Response_label <- factor(
  baseline_oof$Response,
  levels = cv_responses,
  labels = unname(baseline_labels[cv_responses])
)

baseline_oof$Re_factor <- factor(
  baseline_oof$Re,
  levels = levels(train$Re_factor)
)

baseline_oof$Fr_factor <- factor(
  baseline_oof$Fr,
  levels = levels(train$Fr_factor)
)

ggplot(
  baseline_oof,
  aes(
    x = St,
    y = Error,
    color = Re_factor
  )
) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed",
    color = "grey50"
  ) +
  geom_point(size = 2) +
  scale_color_manual(values = Re_colors) +
  scale_x_log10(
    breaks = c(0.05, 0.1, 0.3, 1, 3)
  ) +
  facet_grid(
    Response_label ~ Fr_factor,
    scales = "free_y",
    labeller = label_both
```

```
  ) +
  labs(
    title = "Held-out prediction errors from the baseline models",
    x = "Stokes number (log10 scale)",
    y = "Observed minus predicted",
    color = "Re"
  ) +
  theme_bw(base_size = 10) +
  theme(legend.position = "top")
```




```
# Inspect the most influential observations in the full-training fits
baseline_influence <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    cook <- cooks.distance(baseline_models[[response]])
    selected_rows <- head(order(cook, decreasing = TRUE), 2)

    data.frame(
      Response = baseline_labels[[response]],
      Row = train$row_id[selected_rows],
      Re = train$Re[selected_rows],
```

```
      Fr = train$Fr[selected_rows],
      St = train$St[selected_rows],
      Cooks_distance = unname(cook[selected_rows])
    )
  })
)

knitr::kable(
  baseline_influence,
  digits = 4,
  row.names = FALSE,
  caption = "Two largest Cook's distances for each baseline model"
)
```

Table 14: Two largest Cook's distances for each baseline model

Response	Row	Re	Fr	St	Cooks_distance
Mean	2	224	0.052	3.00	0.1172
Mean	51	90	Inf	3.00	0.1081
Standard deviation	80	90	0.052	3.00	0.1881
Standard deviation	58	90	0.052	2.00	0.0957
Skewness	82	90	0.052	0.05	0.1338
Skewness	34	398	Inf	0.05	0.1110
Kurtosis	34	398	Inf	0.05	0.1783
Kurtosis	82	90	0.052	0.05	0.1292

```
#| label: flexible-model-setup
#| message: false

library(glmnet)
```

```
Warning: package 'glmnet' was built under R version 4.5.2
```

```
Loading required package: Matrix
```

```
Loaded glmnet 5.1
```

```r
# Right-hand sides for the unpenalized log-response models
flexible_rhs <- c(
  Log_main = "Re + Fr_factor + St",

  Log_inputs =
    "log10_Re + Fr_factor + log10_St",

  Factor_Re =
    "Re_factor + Fr_factor + log10_St",

  Regime_linear =
    "regime + log10_St",

  Regime_quadratic =
    "regime + log10_St + I(log10_St^2)",

  Regime_curves =
    "regime * (log10_St + I(log10_St^2))",

  Spline =
    "regime + splines::ns(log10_St, df = 3)"
)

# Ridge and lasso use the same expanded predictor set
penalty_formula <- ~ regime * (log10_St + I(log10_St^2))

candidate_names <- c(names(flexible_rhs), "Ridge", "Lasso")

# Create balanced, regime-stratified inner folds
make_inner_folds <- function(groups, K = 5L, seed = 325L) {

  set.seed(seed)

  groups <- droplevels(factor(groups))
  fold_id <- integer(length(groups))
  fold_order <- sample(seq_len(K))
  offset <- 0L

  for (g in levels(groups)) {

    rows <- which(groups == g)
```

```
    assignments <- fold_order[
      ((seq_along(rows) - 1L + offset) %% K) + 1L
    ]

    fold_id[rows] <- assignments[sample.int(length(rows))]
    offset <- offset + length(rows)
  }

  fold_id
}

# Save identical inner splits for all responses and candidates
inner_fold_ids <- lapply(seq_len(K), function(k) {

  analysis_rows <- cv_folds[[k]]$analysis

  make_inner_folds(
    train$regime[analysis_rows],
    seed = 325L + k
  )
})

knitr::kable(
  data.frame(
    Model = names(flexible_rhs),
    Predictors = unname(flexible_rhs)
  ),
  row.names = FALSE,
  caption = "Unpenalized log-response candidate models"
)
```

Table 15: Unpenalized log-response candidate models

Model	Predictors
Log_main	Re + Fr_factor + St
Log_inputs	log10_Re + Fr_factor + log10_St
Factor_Re	Re_factor + Fr_factor + log10_St
Regime_linear	regime + log10_St
Regime_quadratic	regime + log10_St + I(log10_St^2)
Regime_curves	regime * (log10_St + I(log10_St^2))
Spline	regime + splines::ns(log10_St, df = 3)

```r
#| label: flexible-model-functions

fit_flexible_model <- function(
    model_name, response, analysis_data, inner_id = NULL) {

  analysis_data$.log_y <- log10(analysis_data[[response]])

  if (model_name %in% names(flexible_rhs)) {

    model_formula <- reformulate(
      flexible_rhs[[model_name]],
      response = ".log_y"
    )

    fit <- lm(model_formula, data = analysis_data)

    stopifnot(
      fit$rank == ncol(model.matrix(fit))
    )

    fitted_log <- as.numeric(fitted(fit))

    object <- list(
      engine = "lm",
      fit = fit,
      lambda = NA_real_
    )

  } else {

    stopifnot(
      model_name %in% c("Ridge", "Lasso"),
      length(inner_id) == nrow(analysis_data)
    )

    # Exclude the intercept: glmnet supplies its own unpenalized intercept
    x <- model.matrix(
      penalty_formula,
      data = analysis_data
    )[, -1, drop = FALSE]

    alpha_value <- if (model_name == "Ridge") 0 else 1
```

```
    fit <- glmnet::cv.glmnet(
      x = x,
      y = analysis_data$.log_y,
      family = "gaussian",
      alpha = alpha_value,
      foldid = inner_id,
      type.measure = "mse",
      standardize = TRUE
    )

    fitted_log <- as.numeric(
      predict(fit, newx = x, s = "lambda.min")
    )

    object <- list(
      engine = "glmnet",
      fit = fit,
      columns = colnames(x),
      lambda = fit$lambda.min
    )
  }

  log_residuals <- analysis_data$.log_y - fitted_log

  object$smearing <- mean(10^log_residuals)

  stopifnot(
    is.finite(object$smearing),
    object$smearing > 0
  )

  object
}

predict_flexible_model <- function(object, new_data) {

  if (object$engine == "lm") {

    predicted_log <- as.numeric(
      predict(object$fit, newdata = new_data)
    )
```

```
  } else {

    new_x <- model.matrix(
      penalty_formula,
      data = new_data
    )[, -1, drop = FALSE]

    new_x <- new_x[, object$columns, drop = FALSE]

    predicted_log <- as.numeric(
      predict(
        object$fit,
        newx = new_x,
        s = "lambda.min"
      )
    )
  }

  predicted <- 10^predicted_log * object$smearing

  # Do not silently clip or discard problematic predictions
  stopifnot(
    all(is.finite(predicted)),
    all(predicted > 0)
  )

  predicted
}
```

```
#| label: flexible-model-cross-validation
#| message: false

flexible_oof_parts <- list()
part_number <- 1L

for (response in cv_responses) {

  for (model_name in candidate_names) {

    for (k in seq_len(K)) {

      analysis_rows <- cv_folds[[k]]$analysis
```

```
      assessment_rows <- cv_folds[[k]]$assessment

      analysis_data <- train[analysis_rows, ]
      assessment_data <- train[assessment_rows, ]

      model <- fit_flexible_model(
        model_name = model_name,
        response = response,
        analysis_data = analysis_data,
        inner_id = inner_fold_ids[[k]]
      )

      predictions <- predict_flexible_model(
        model,
        assessment_data
      )

      flexible_oof_parts[[part_number]] <- data.frame(
        row_id = assessment_data$row_id,
        Response = response,
        Model = model_name,
        Fold = k,
        Observed = assessment_data[[response]],
        Predicted = predictions,
        Lambda = model$lambda
      )

      part_number <- part_number + 1L
    }
  }
}

flexible_oof <- do.call(rbind, flexible_oof_parts)

cat(
  "Completed outer cross-validation for",
  length(candidate_names), "candidates and",
  length(cv_responses), "responses.\n"
)
```

```
Completed outer cross-validation for 9 candidates and 4 responses.
```

```r
#| label: collect-model-comparisons

# Original-scale linear baseline from Step 6
original_linear_oof <- data.frame(
  row_id = baseline_oof$row_id,
  Response = baseline_oof$Response,
  Model = "Original_linear",
  Fold = baseline_oof$Fold,
  Observed = baseline_oof$Observed,
  Predicted = baseline_oof$Predicted,
  Lambda = NA_real_
)

# Training-fold mean benchmark from Step 6
mean_only_oof <- data.frame(
  row_id = baseline_oof$row_id,
  Response = baseline_oof$Response,
  Model = "Mean_only",
  Fold = baseline_oof$Fold,
  Observed = baseline_oof$Observed,
  Predicted = baseline_oof$Mean_only_prediction,
  Lambda = NA_real_
)

all_model_oof <- rbind(
  mean_only_oof,
  original_linear_oof,
  flexible_oof
)

model_order <- c(
  "Mean_only",
  "Original_linear",
  candidate_names
)

comparison_parts <- list()
part_number <- 1L

for (response in cv_responses) {

  for (model_name in model_order) {
```

```
    predictions <- all_model_oof[
      all_model_oof$Response == response &
        all_model_oof$Model == model_name,
    ]

    stopifnot(
      nrow(predictions) == nrow(train),
      identical(
        sort(predictions$row_id),
        sort(train$row_id)
      )
    )

    scores <- score_predictions(
      predictions$Observed,
      predictions$Predicted
    )

    comparison_parts[[part_number]] <- data.frame(
      Response = response,
      Model = model_name,
      CV_RMSE = scores$RMSE,
      CV_MAE = scores$MAE,
      Nonpositive = scores$Nonpositive_predictions
    )

    part_number <- part_number + 1L
  }
}

model_comparison <- do.call(rbind, comparison_parts)

# Print one manageable table per response, in progression order
for (response in cv_responses) {

  result_table <- model_comparison[
    model_comparison$Response == response,
    c("Model", "CV_RMSE", "CV_MAE", "Nonpositive")
  ]

  print(
    knitr::kable(
```

```
      result_table,
      digits = 4,
      row.names = FALSE,
      caption = paste(
        "Original-scale validation results:",
        baseline_labels[[response]]
      )
    )
  )
}
```

Table: Original-scale validation results: Mean

Model	CV_RMSE	CV_MAE	Nonpositive
Mean_only	0.0557	0.0501	0
Original_linear	0.0344	0.0297	19
Log_main	0.0213	0.0120	0
Log_inputs	0.0109	0.0057	0
Factor_Re	0.0107	0.0057	0
Regime_linear	0.0092	0.0042	0
Regime_quadratic	0.0089	0.0040	0
Regime_curves	0.0087	0.0031	0
Spline	0.0094	0.0040	0
Ridge	0.0275	0.0120	0
Lasso	0.0105	0.0050	0

Table: Original-scale validation results: Standard deviation

Model	CV_RMSE	CV_MAE	Nonpositive
Mean_only	8.9166	5.7280	0
Original_linear	6.9111	5.2599	23
Log_main	6.5332	2.7071	0
Log_inputs	5.6960	2.3876	0
Factor_Re	5.7253	2.3976	0
Regime_linear	3.3865	1.1428	0
Regime_quadratic	1.7338	0.6561	0
Regime_curves	3.3367	1.0908	0

Spline	1.5621	0.5243	0
Ridge	6.0250	2.2718	0
Lasso	3.4351	1.2109	0

Table: Original-scale validation results: Skewness

Model	CV_RMSE	CV_MAE	Nonpositive
Mean_only	112.7479	107.4515	0
Original_linear	55.3672	43.8728	1
Log_main	164.1616	94.1407	0
Log_inputs	149.3610	97.3659	0
Factor_Re	139.8251	99.3731	0
Regime_linear	39.4631	18.6780	0
Regime_quadratic	39.1864	19.0416	0
Regime_curves	48.0038	16.4034	0
Spline	38.8431	18.3799	0
Ridge	51.5856	22.8938	0
Lasso	47.0845	18.7108	0

Table: Original-scale validation results: Kurtosis

Model	CV_RMSE	CV_MAE	Nonpositive
Mean_only	38081.13	35489.359	0
Original_linear	21207.66	16246.716	7
Log_main	193261.25	80819.654	0
Log_inputs	162562.79	79397.997	0
Factor_Re	141468.85	77750.490	0
Regime_linear	18204.45	7623.177	0
Regime_quadratic	18820.96	8604.257	0
Regime_curves	21650.78	7462.564	0
Spline	18950.04	8495.020	0
Ridge	23752.16	10144.876	0
Lasso	22992.21	8801.292	0

```
#| label: store-full-training-models
#| message: false

full_inner_id <- make_inner_folds(
```

```
  train$regime,
  seed = 1325L
)

flexible_models <- setNames(
  lapply(cv_responses, function(response) {

    setNames(
      lapply(candidate_names, function(model_name) {

        fit_flexible_model(
          model_name = model_name,
          response = response,
          analysis_data = train,
          inner_id = full_inner_id
        )

      }),
      candidate_names
    )

  }),
  cv_responses
)

cat(
  "Stored full-training candidate fits.",
  "No test predictions have been generated.\n"
)
```

```
Stored full-training candidate fits. No test predictions have been generated.
```

```
#| label: rank-candidate-models

ranked_comparison <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    results <- model_comparison[
      model_comparison$Response == response,
    ]
```

```
    results <- results[
      order(results$CV_RMSE, results$CV_MAE, results$Model),
    ]

    results$Rank <- seq_len(nrow(results))

    results$Gap_percent <- 100 * (
      results$CV_RMSE / min(results$CV_RMSE) - 1
    )

    rownames(results) <- NULL
    results
  })
)

# Keep the three best flexible candidates for each response
shortlist <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    results <- ranked_comparison[
      ranked_comparison$Response == response &
        ranked_comparison$Model %in% candidate_names,
    ]

    head(results, 3)
  })
)

# Provisional choices: minimum original-scale CV RMSE
provisional_model_names <- setNames(
  vapply(cv_responses, function(response) {

    results <- shortlist[shortlist$Response == response, ]
    results$Model[1]

  }, character(1)),
  cv_responses
)

for (response in cv_responses) {
```

```
  results <- ranked_comparison[
    ranked_comparison$Response == response,
    c("Rank", "Model", "CV_RMSE", "CV_MAE", "Gap_percent")
  ]

  print(
    knitr::kable(
      results,
      digits = 5,
      row.names = FALSE,
      caption = paste(
        "Model ranking:",
        baseline_labels[[response]]
      )
    )
  )
}
```

Table: Model ranking: Mean

Rank	Model	CV_RMSE	CV_MAE	Gap_percent
1	Regime_curves	0.00872	0.00310	0.00000
2	Regime_quadratic	0.00889	0.00403	1.97666
3	Regime_linear	0.00917	0.00416	5.14621
4	Spline	0.00936	0.00396	7.37918
5	Lasso	0.01048	0.00499	20.21852
6	Factor_Re	0.01069	0.00566	22.57333
7	Log_inputs	0.01094	0.00572	25.48249
8	Log_main	0.02131	0.01195	144.37020
9	Ridge	0.02745	0.01196	214.76705
10	Original_linear	0.03438	0.02971	294.16771
11	Mean_only	0.05567	0.05006	538.34072

Table: Model ranking: Standard deviation

Rank	Model	CV_RMSE	CV_MAE	Gap_percent
1	Spline	1.56214	0.52434	0.00000

2	Regime_quadratic	1.73380	0.65607	10.98916
3	Regime_curves	3.33668	1.09085	113.59710
4	Regime_linear	3.38646	1.14277	116.78371
5	Lasso	3.43509	1.21087	119.89647
6	Log_inputs	5.69604	2.38759	264.63134
7	Factor_Re	5.72531	2.39762	266.50503
8	Ridge	6.02495	2.27176	285.68630
9	Log_main	6.53315	2.70709	318.21855
10	Original_linear	6.91114	5.25986	342.41570
11	Mean_only	8.91659	5.72803	470.79385

Table: Model ranking: Skewness

Rank	Model	CV_RMSE	CV_MAE	Gap_percent
1	Spline	38.84305	18.37989	0.00000
2	Regime_quadratic	39.18637	19.04155	0.88386
3	Regime_linear	39.46311	18.67800	1.59630
4	Lasso	47.08445	18.71084	21.21718
5	Regime_curves	48.00383	16.40345	23.58409
6	Ridge	51.58564	22.89383	32.80532
7	Original_linear	55.36720	43.87285	42.54080
8	Mean_only	112.74793	107.45150	190.26536
9	Factor_Re	139.82513	99.37306	259.97460
10	Log_inputs	149.36104	97.36591	284.52447
11	Log_main	164.16161	94.14074	322.62797

Table: Model ranking: Kurtosis

Rank	Model	CV_RMSE	CV_MAE	Gap_percent
1	Regime_linear	18204.45	7623.177	0.00000
2	Regime_quadratic	18820.96	8604.257	3.38659
3	Spline	18950.04	8495.020	4.09565
4	Original_linear	21207.66	16246.716	16.49715
5	Regime_curves	21650.78	7462.564	18.93126
6	Lasso	22992.21	8801.292	26.29999
7	Ridge	23752.16	10144.876	30.47448
8	Mean_only	38081.13	35489.359	109.18588
9	Factor_Re	141468.85	77750.490	677.11148
10	Log_inputs	162562.79	79397.997	792.98393

```
|   11|Log_main          | 193261.25| 80819.654|   961.61563|
```

```
#| label: compare-errors-by-setting
#| fig-width: 9
#| fig-height: 8

# Retain held-out predictions for shortlisted candidates
comparison_oof <- merge(
  all_model_oof,
  shortlist[c("Response", "Model")],
  by = c("Response", "Model"),
  sort = FALSE
)

# Attach simulation settings using the stable row IDs
metadata_rows <- match(comparison_oof$row_id, train$row_id)
stopifnot(!anyNA(metadata_rows))

comparison_oof$Re <- train$Re[metadata_rows]
comparison_oof$Fr <- train$Fr[metadata_rows]
comparison_oof$St <- train$St[metadata_rows]
comparison_oof$regime <- train$regime[metadata_rows]

comparison_oof$St_range <- ifelse(
  comparison_oof$St <= 0.3,
  "St <= 0.3",
  "St > 0.3"
)

comparison_oof$SE <- (
  comparison_oof$Observed - comparison_oof$Predicted
)^2

comparison_oof$AE <- abs(
  comparison_oof$Observed - comparison_oof$Predicted
)

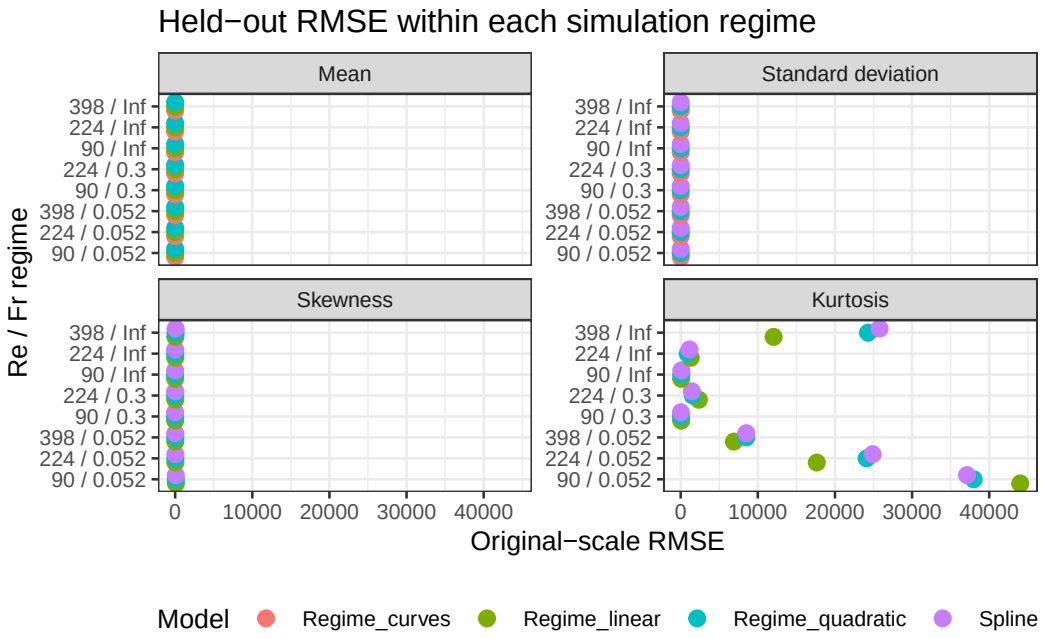
# Error within each simulation regime
regime_scores <- aggregate(
  cbind(SE, AE) ~ Response + Model + regime,
  data = comparison_oof,
  FUN = mean
```

```
)

regime_scores$RMSE <- sqrt(regime_scores$SE)
regime_scores$MAE <- regime_scores$AE

regime_scores$Response_label <- factor(
  regime_scores$Response,
  levels = cv_responses,
  labels = unname(baseline_labels[cv_responses])
)

ggplot(
  regime_scores,
  aes(x = regime, y = RMSE, color = Model)
) +
  geom_point(
    position = position_dodge(width = 0.6),
    size = 2.5
  ) +
  coord_flip() +
  facet_wrap(
    ~ Response_label,
    scales = "free_y",
    ncol = 2
  ) +
  labs(
    title = "Held-out RMSE within each simulation regime",
    x = "Re / Fr regime",
    y = "Original-scale RMSE",
    color = "Model"
  ) +
  theme_bw(base_size = 10) +
  theme(legend.position = "bottom")
```




```r
# Error within the two St ranges
St_scores <- aggregate(
  cbind(SE, AE) ~ Response + Model + St_range,
  data = comparison_oof,
  FUN = mean
)

St_scores$RMSE <- sqrt(St_scores$SE)
St_scores$MAE <- St_scores$AE

St_counts <- aggregate(
  row_id ~ Response + Model + St_range,
  data = comparison_oof,
  FUN = length
)

names(St_counts)[names(St_counts) == "row_id"] <- "N"

St_scores <- merge(
  St_scores,
  St_counts,
  by = c("Response", "Model", "St_range")
)
```

```r
# Display the St-range results for provisional selections
selection_map <- data.frame(
  Response = names(provisional_model_names),
  Model = unname(provisional_model_names)
)

selected_St_scores <- merge(
  St_scores,
  selection_map,
  by = c("Response", "Model")
)

selected_St_scores$Response <- unname(
  baseline_labels[selected_St_scores$Response]
)

knitr::kable(
  selected_St_scores[
    c("Response", "Model", "St_range", "N", "RMSE", "MAE")
  ],
  digits = 5,
  row.names = FALSE,
  caption = "Validation performance by St range: provisional selections"
)
```

Table 16: Validation performance by St range: provisional selections

Response	Model	St_range	N	RMSE	MAE
Kurtosis	Regime_linear	St <= 0.3	26	31355.32064	14753.55736
Kurtosis	Regime_linear	St > 0.3	63	7900.88273	4680.47994
Mean	Regime_curves	St <= 0.3	26	0.00753	0.00365
Mean	Regime_curves	St > 0.3	63	0.00917	0.00288
Standard deviation	Spline	St <= 0.3	26	1.73224	0.73671
Standard deviation	Spline	St > 0.3	63	1.48627	0.43669
Skewness	Spline	St <= 0.3	26	67.00457	36.75104
Skewness	Spline	St > 0.3	63	16.69128	10.79814

```r
#| label: inspect-largest-errors

worst_error_tables <- setNames(
```

```r
lapply(cv_responses, function(response) {

  # Include problematic simpler models for the shape responses
  extra_models <- if (
    response %in% c("skewness_volume", "kurtosis_volume")
  ) {
    c("Log_main", "Factor_Re")
  } else {
    character(0)
  }

  models_to_inspect <- unique(c(
    shortlist$Model[shortlist$Response == response],
    extra_models
  ))

  do.call(
    rbind,
    lapply(models_to_inspect, function(model_name) {

      predictions <- all_model_oof[
        all_model_oof$Response == response &
          all_model_oof$Model == model_name,
      ]

      squared_errors <- (
        predictions$Observed - predictions$Predicted
      )^2

      worst <- which.max(squared_errors)
      total_error <- sum(squared_errors)

      data.frame(
        Model = model_name,
        Row = predictions$row_id[worst],
        Observed = predictions$Observed[worst],
        Predicted = predictions$Predicted[worst],
        SSE_share_percent = if (total_error > 0) {
          100 * squared_errors[worst] / total_error
        } else {
          0
        }
```

```
        )
      })
    )
  }),
  cv_responses
)

for (response in cv_responses) {

  print(
    knitr::kable(
      worst_error_tables[[response]],
      digits = 5,
      row.names = FALSE,
      caption = paste(
        "Largest individual prediction errors:",
        baseline_labels[[response]]
      )
    )
  )
}
```

Table: Largest individual prediction errors: Mean

Model	Row	Observed	Predicted	SSE_share_percent
Regime_curves	51	0.17234	0.10755	62.01517
Regime_quadratic	51	0.17234	0.13052	24.84124
Regime_linear	51	0.17234	0.12404	31.17224

Table: Largest individual prediction errors: Standard deviation

Model	Row	Observed	Predicted	SSE_share_percent
Spline	80	32.31526	41.35630	37.63638
Regime_quadratic	20	22.75360	14.04779	28.32882
Regime_curves	80	32.31526	14.77430	31.05184

Table: Largest individual prediction errors: Skewness

Model	Row	Observed	Predicted	SSE_share_percent
Spline	82	57.71417	298.9154	43.32537
Regime_quadratic	82	57.71417	304.1901	44.45175
Regime_linear	82	57.71417	347.0922	60.41692
Log_main	27	312.36435	849.4588	12.02730
Factor_Re	27	312.36435	686.3652	8.03868

Table: Largest individual prediction errors: Kurtosis

Model	Row	Observed	Predicted	SSE_share_percent
Regime_linear	82	5622.342	136985.9	58.50655
Regime_quadratic	82	5622.342	115247.9	38.11977
Spline	82	5622.342	114269.5	36.93403
Log_main	27	97635.239	903873.3	19.55451
Factor_Re	27	97635.239	680439.9	19.06928

```
#| label: diagnose-shortlisted-models
#| fig-width: 8
#| fig-height: 7

plot_candidate_diagnostics <- function(
    object, response, model_name) {

  old_par <- par(
    mfrow = c(2, 2),
    oma = c(0, 0, 3, 0),
    mar = c(5, 4, 2, 1)
  )

  on.exit(par(old_par))

  # Remove the smearing correction to recover fitted log values
  fitted_log <- log10(
    predict_flexible_model(object, train) / object$smearing
  )

  log_residuals <- log10(train[[response]]) - fitted_log
```

```r
# Residuals versus fitted log response
plot(
  fitted_log, log_residuals,
  xlab = "Fitted log10(response)",
  ylab = "Log-scale residual",
  main = "Residuals vs fitted"
)

abline(h = 0, lty = 2, col = "grey50")
lines(lowess(fitted_log, log_residuals), col = "red")

# Normal Q-Q plot
qqnorm(
  log_residuals,
  main = "Normal Q-Q: log-scale residuals"
)

qqline(log_residuals, col = "red")

# Remaining St structure
plot(
  train$log10_St, log_residuals,
  xlab = "log10(St)",
  ylab = "Log-scale residual",
  main = "Residuals vs log10(St)"
)

abline(h = 0, lty = 2, col = "grey50")
lines(lowess(train$log10_St, log_residuals), col = "red")

# Differences between simulation regimes
boxplot(
  log_residuals ~ train$regime,
  las = 2,
  cex.axis = 0.65,
  xlab = "",
  ylab = "Log-scale residual",
  main = "Residuals by Re / Fr regime"
)

abline(h = 0, lty = 2, col = "grey50")
```

```
  mtext(
    paste(baseline_labels[[response]], ":", model_name),
    outer = TRUE,
    line = 1,
    font = 2
  )
}

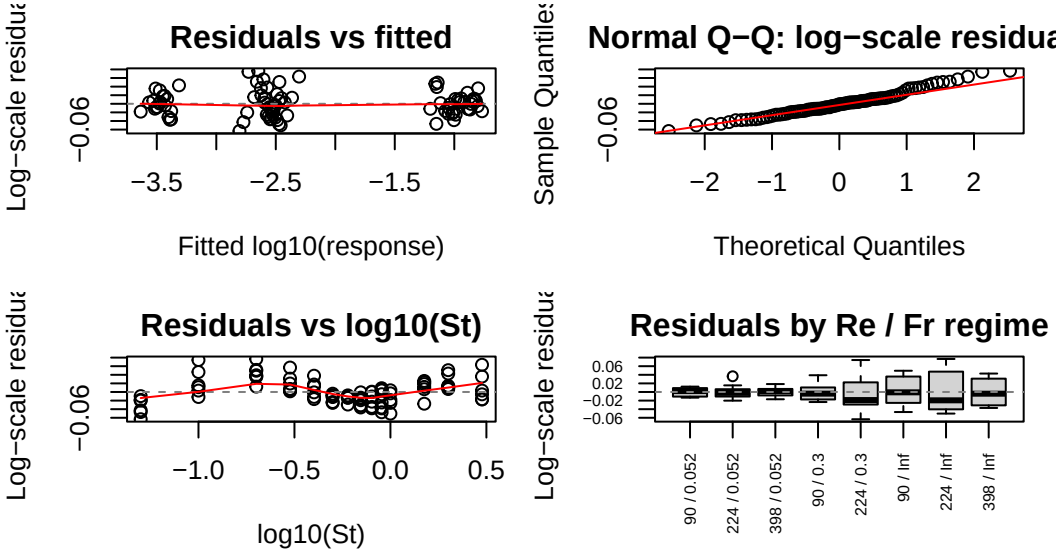
for (response in cv_responses) {

  models_to_diagnose <- head(
    shortlist$Model[shortlist$Response == response],
    2
  )

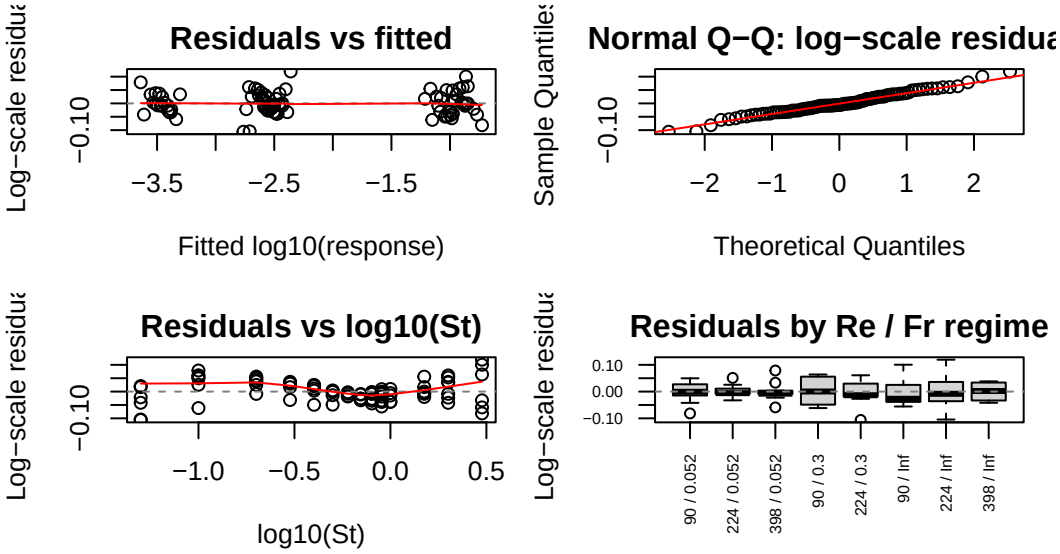
  for (model_name in models_to_diagnose) {

    plot_candidate_diagnostics(
      flexible_models[[response]][[model_name]],
      response,
      model_name
    )
  }
}
```

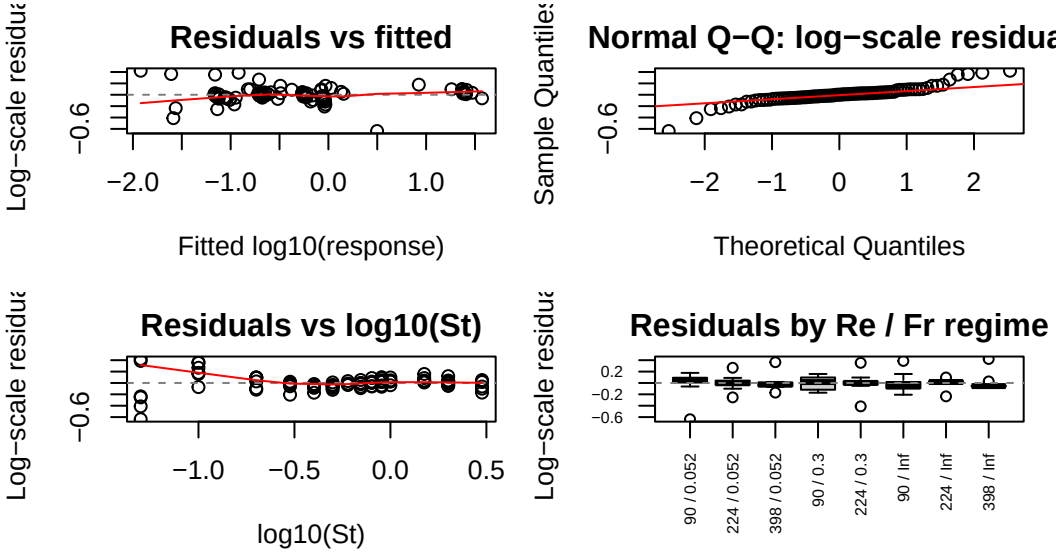
Mean : Regime_curves



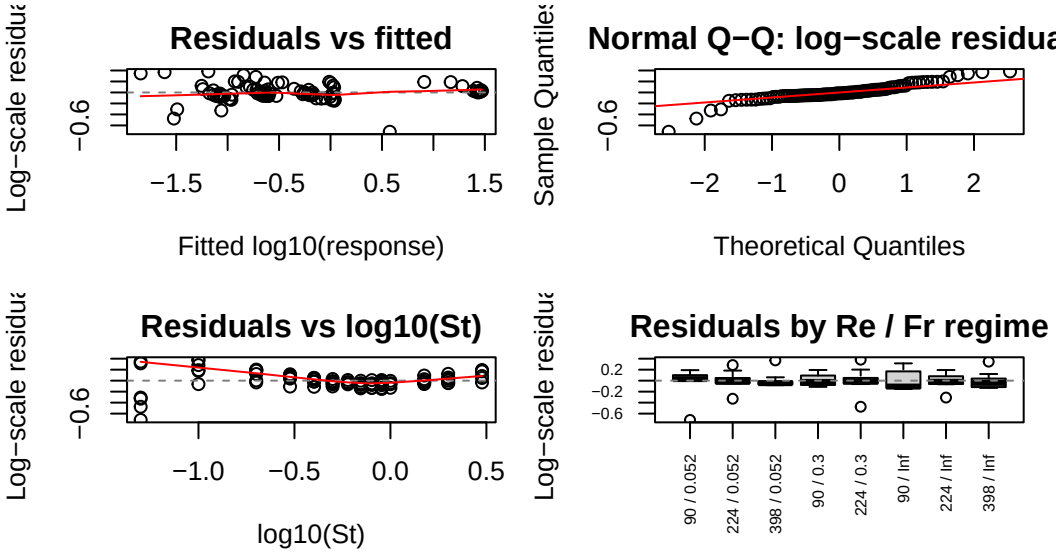
Mean : Regime_quadratic



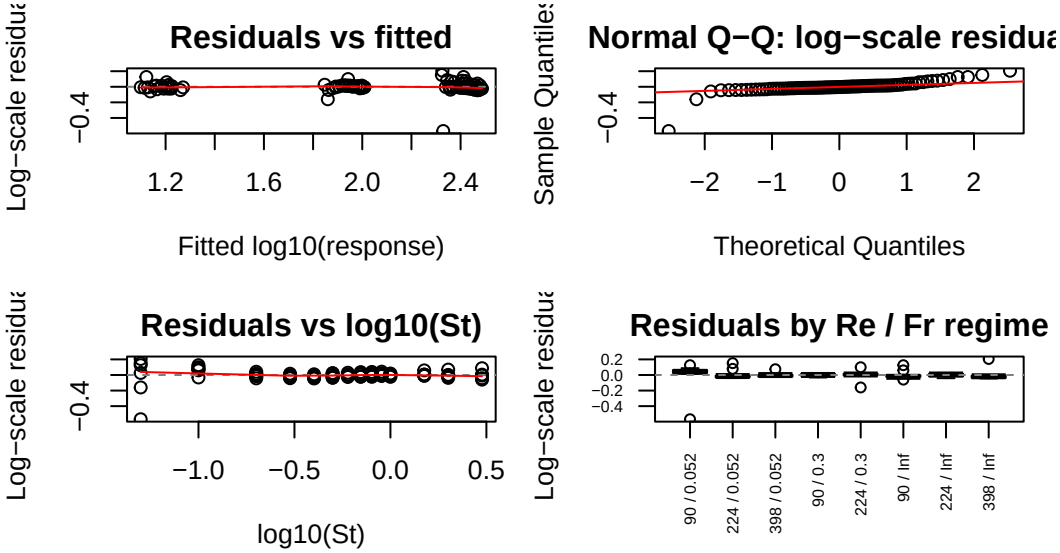
Standard deviation : Spline



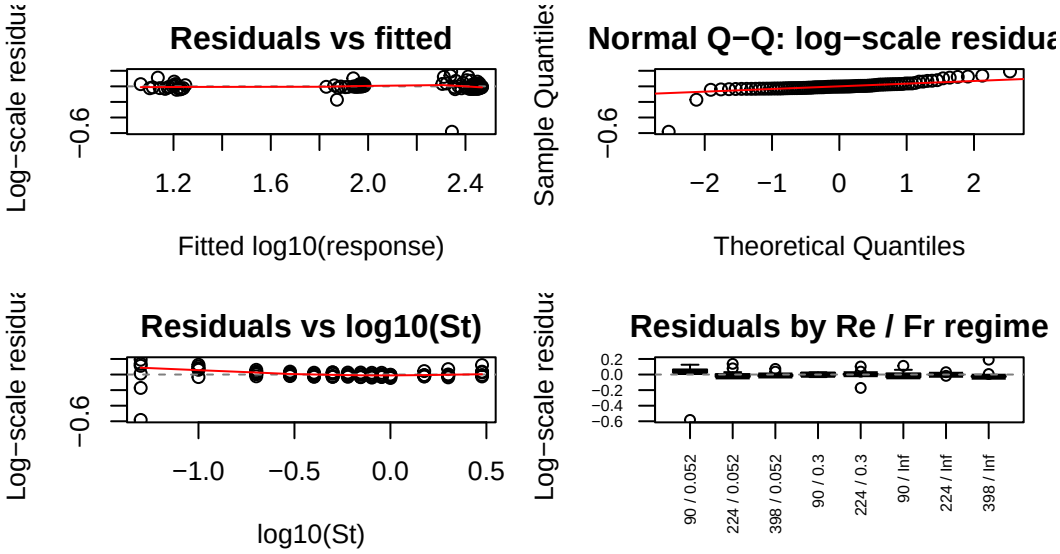
Standard deviation : Regime_quadratic

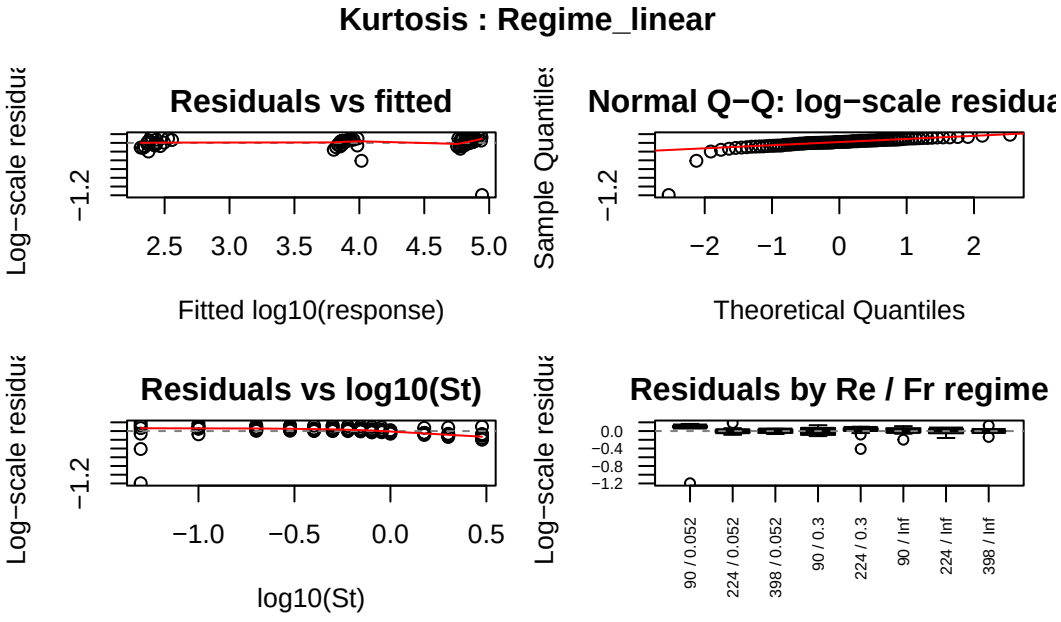


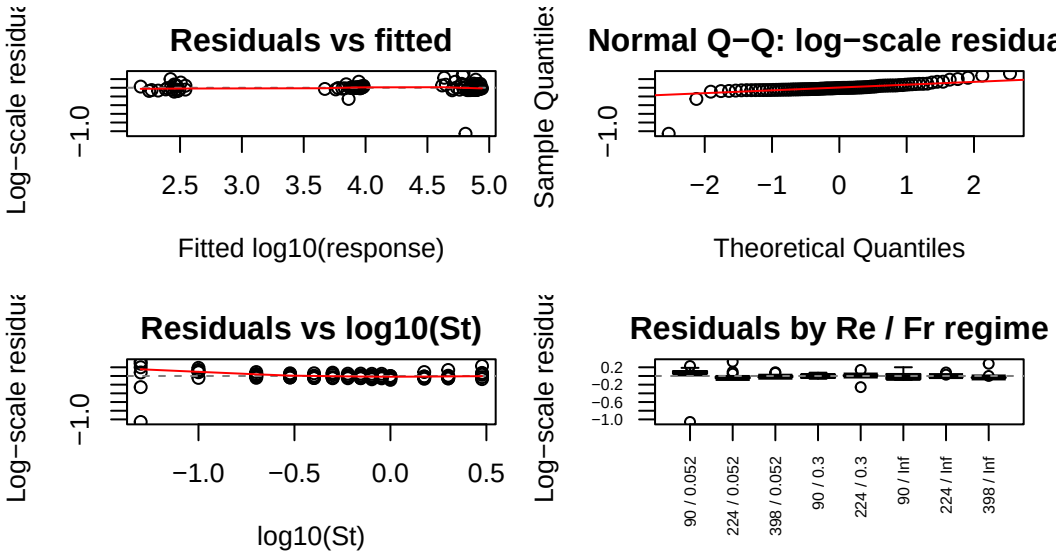
Skewness : Spline



Skewness : Regime_quadratic








```
#| label: store-provisional-selections

selected_model_names <- provisional_model_names
```

```r
selected_models <- setNames(
  lapply(cv_responses, function(response) {

    flexible_models[[response]][[
      selected_model_names[[response]]
    ]]

  }),
  cv_responses
)

# Preserve the corresponding held-out predictions
selected_oof <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    all_model_oof[
      all_model_oof$Response == response &
        all_model_oof$Model == selected_model_names[[response]],
    ]
  })
)

selection_summary <- do.call(
  rbind,
  lapply(cv_responses, function(response) {

    chosen <- model_comparison[
      model_comparison$Response == response &
        model_comparison$Model == selected_model_names[[response]],
    ]

    baseline_rmse <- model_comparison$CV_RMSE[
      model_comparison$Response == response &
        model_comparison$Model == "Original_linear"
    ]

    data.frame(
      Response = baseline_labels[[response]],
      Model = chosen$Model,
      CV_RMSE = chosen$CV_RMSE,
      CV_MAE = chosen$CV_MAE,
```

```
      RMSE_reduction_percent = 100 * (
        1 - chosen$CV_RMSE / baseline_rmse
      )
    )
  })
)

knitr::kable(
  selection_summary,
  digits = 5,
  row.names = FALSE,
  caption = "Provisional selections and improvement over the original linear baseline"
)
```

Table 17: Provisional selections and improvement over the original linear baseline

Response	Model	CV_RMSE	CV_MAE	RMSE_reduction_percent
Mean	Regime_curves	0.00872	0.00310	74.63009
Standard deviation	Spline	1.56214	0.52434	77.39682
Skewness	Spline	38.84305	18.37989	29.84465
Kurtosis	Regime_linear	18204.44697	7623.17671	14.16099